



# VERIQTA

## PRODUCTION HANDBOOK

ENTERPRISE  
EDITION

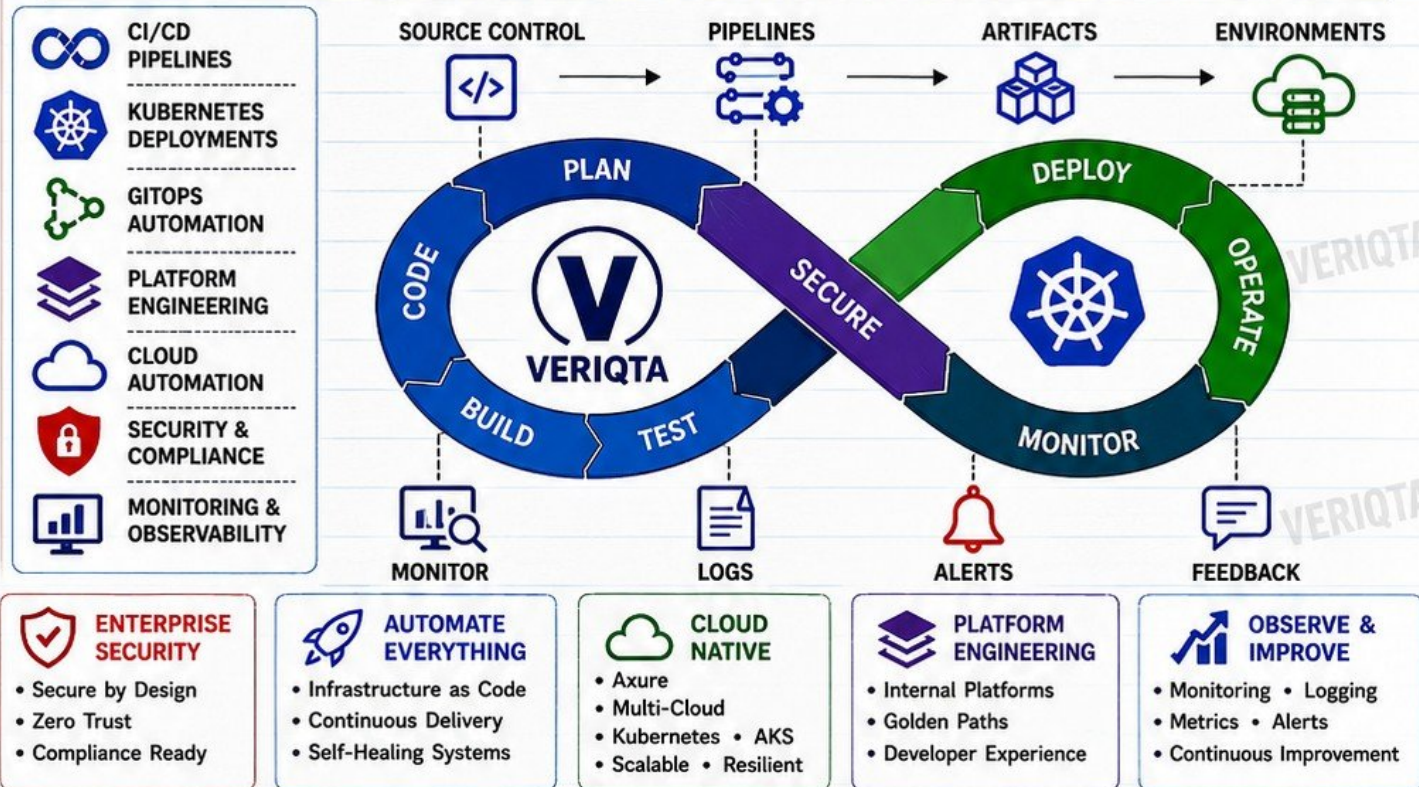


# ENTERPRISE AZURE DEVOPS

FOR  
CI/CD, KUBERNETES, GITOPS,  
PLATFORM ENGINEERING,  
AND CLOUD AUTOMATION

DESIGN • BUILD • DEPLOY • SECURE • OPERATE • MONITOR

BUILD SCALABLE • SECURE • AUTOMATED • RESILIENT SYSTEMS AT ENTERPRISE SCALE



VERIQTA



FROM CODE TO CLOUD. FROM IDEA TO IMPACT.  
AUTOMATE • SCALE • SECURE • DELIVER

# 1. ENTERPRISE CI/CD ARCHITECTURE



## MODERN CI/CD ARCHITECTURE

- Automate build, test, security, and deploy
- Shift-left quality and security
- Continuous feedback and improvement
- Standardized, reusable, and scalable



## ENTERPRISE PIPELINE DESIGN

- Multi-stage, modular, and reusable pipelines
- Templates and shared components
- Environment promotion and approvals
- Governance, compliance, and auditability



## PIPELINE LIFECYCLE

- Plan → Code → Build → Test → Deploy → Operate
- Continuous visibility and feedback
- Automated quality and security gates
- Measure, learn, and improve continuously



## RELEASE ORCHESTRATION

- Coordinate deployments across environments
- Approval workflows and quality gates
- Progressive delivery and rollback
- Track releases, versions, and changes



## CONTINUOUS DELIVERY vs CONTINUOUS DEPLOYMENT

- Continuous Delivery (CD): Always releasable
- Continuous Deployment (CD): Auto-release
- Choose the right model for the organization
- Balance speed, quality, and risk



## ENTERPRISE DEVOPS MATURITY

- Level 1: Manual and siloed
- Level 2: Basic automation
- Level 3: Standardized and measurable
- Level 4: Integrated and scalable
- Level 5: Optimized and innovative



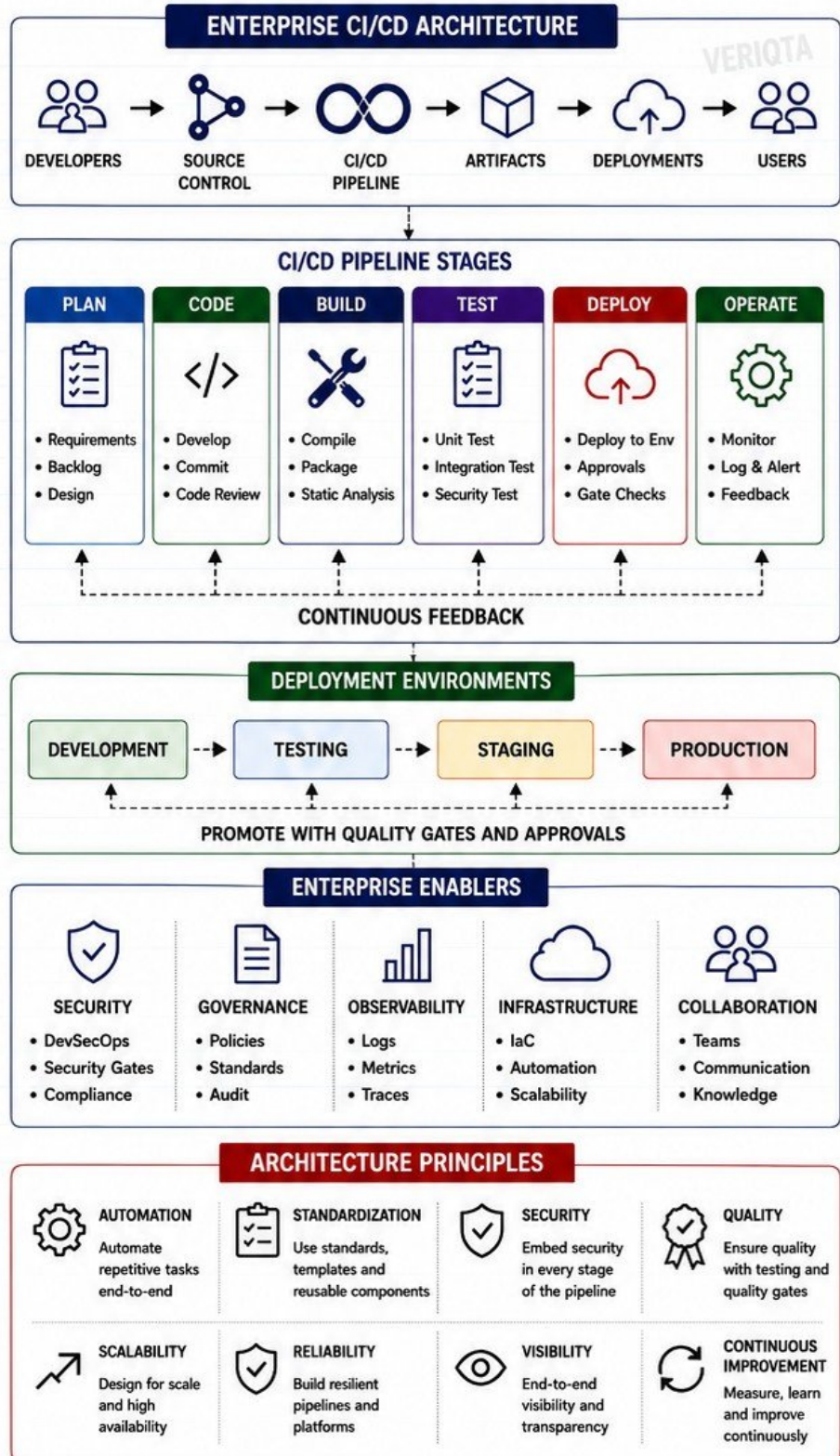
## PLATFORM ENGINEERING INTEGRATION

- Self-service developer platforms
- Golden paths and reusable services
- Infrastructure as Code and automation
- Developer experience and productivity



## BEST PRACTICES

- Automate everything, repeatable by design
- Security embedded in every stage
- Monitor, measure, and optimize
- Keep it simple, scalable, and reliable



# 2. ENTERPRISE YAML PIPELINES



## MULTI-STAGE PIPELINES

- Break complex workflows into stages
- Enforce quality gates between stages
- Run stages in sequence or in parallel
- Improve visibility and control



## TEMPLATES

- Use templates to standardize pipelines
- Reduce duplication and maintenance
- Parameterize for flexibility
- Promote consistency across teams



## REUSABLE JOBS

- Encapsulate tasks into reusable jobs
- Share across multiple pipelines
- Improve maintainability
- Enforce standard build and test steps

VERIQTA



## TEMPLATE REPOSITORIES

- Store templates in a central repo
- Version control and audit changes
- Reference templates across projects
- Manage via Git and approvals



## RUNTIME EXPRESSIONS

- Evaluate variables and expressions
- Use `\${}` for compile-time
- Use `\${}` for runtime evaluation
- Build dynamic and intelligent pipelines



## CONDITIONS

- Control stage, job, and step execution
- Use conditions to enforce rules
- Support success(), failed(), etc.
- Build resilient and adaptive pipelines



## ADVANCED VARIABLES

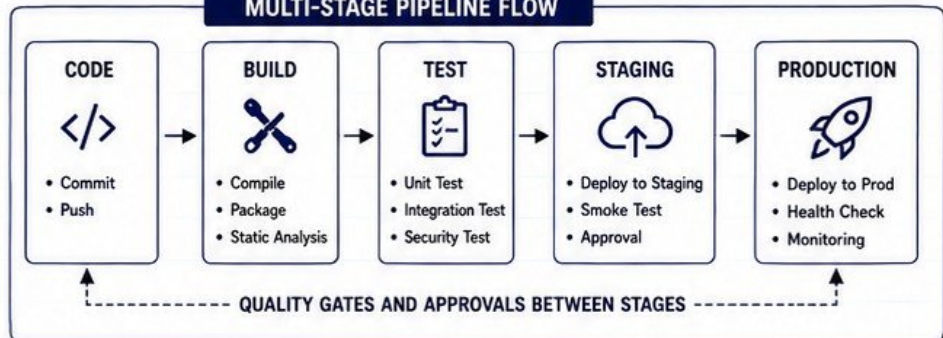
- Variable groups and key vaults
- Output variables between jobs/stages
- Scope variables at different levels
- Secure and dynamic configuration



## ENTERPRISE STANDARDS

- Consistent naming conventions
- Standard templates and guardrails
- Security, compliance, and governance
- Audit, monitor, and improve continuously

## MULTI-STAGE PIPELINE FLOW



## ENTERPRISE YAML EXAMPLE (MULTI-STAGE)

```

1 trigger:
2   branches:
3     include:
4       - main
5
6 variables:
7   buildConfiguration: 'Release'
8   dockerRegistry: 'myACR.azurecr.io'
9
10 stages:
11   - stage: Build
12     displayName: Build and Package
13     jobs:
14       - job: BuildJob
15         pool:
16           vmImage: 'ubuntu-latest'
17         steps:
18           - script: dotnet build --configuration $(buildConfiguration)
19             displayName: 'Build Solution'
20
21   - stage: Test
22     dependsOn: Build
23     condition: succeeded()
24     displayName: Test Stage
  
```

## KEY FEATURES

- ✓ Multi-stage execution
- ✓ Reusable templates
- ✓ Conditions and approvals
- ✓ Runtime expressions
- ✓ Secure variable handling
- ✓ Enterprise ready

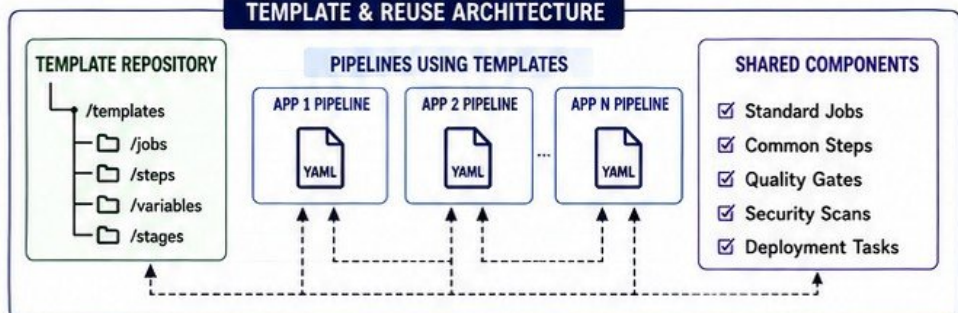
## EXPRESSION EXAMPLES

`${variables.varName}`  
Compile-time expression

`$(variables.varName)`  
Runtime expression

`$(if eq(variables['env'], 'prod'))`  
Conditional insertion

## TEMPLATE & REUSE ARCHITECTURE



## ENTERPRISE YAML PIPELINE STANDARDS

| NAMING                                                                                                                             | MODULARITY                                                                                                                  | SECURITY                                                                                                                        | QUALITY                                                                                                               | OBSERVABILITY                                                                                                                      | GOVERNANCE                                                                                                  |
|------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"> <li>• Consistent names</li> <li>• Clear and meaningful</li> <li>• Easy to understand</li> </ul> | <ul style="list-style-type: none"> <li>• Use templates</li> <li>• Keep pipelines DRY</li> <li>• Reuse components</li> </ul> | <ul style="list-style-type: none"> <li>• Least privilege</li> <li>• Secrets in Key Vault</li> <li>• Secure variables</li> </ul> | <ul style="list-style-type: none"> <li>• Quality gates</li> <li>• Automated tests</li> <li>• Code scanning</li> </ul> | <ul style="list-style-type: none"> <li>• Logs and metrics</li> <li>• Alerts and dashboards</li> <li>• Trace and monitor</li> </ul> | <ul style="list-style-type: none"> <li>• Approvals</li> <li>• Compliance</li> <li>• Audit trails</li> </ul> |



# 3. PIPELINE TEMPLATES



## STAGE TEMPLATES

- Define standard stage structure
- Reuse across multiple pipelines
- Enforce consistency
- Simplify maintenance



## JOB TEMPLATES

- Standardize job configuration
- Reuse job logic and steps
- Improve reliability and speed
- Reduce duplication



## STEP TEMPLATES

- Encapsulate common steps
- Reuse scripts, tools, and tasks
- Promote best practices
- Keep pipelines clean



## VARIABLE TEMPLATES

- Centralize variables
- Manage environments easily
- Secure and reusable
- Reduce hardcoding



## SHARED TEMPLATE REPOSITORIES

- Store templates in Git repos
- Share across teams and projects
- Version controlled
- Access via reference



## TEMPLATE VERSIONING

- Use semantic versioning
- Lock versions for stability
- Allow controlled updates
- Avoid breaking changes



## DRY PIPELINES

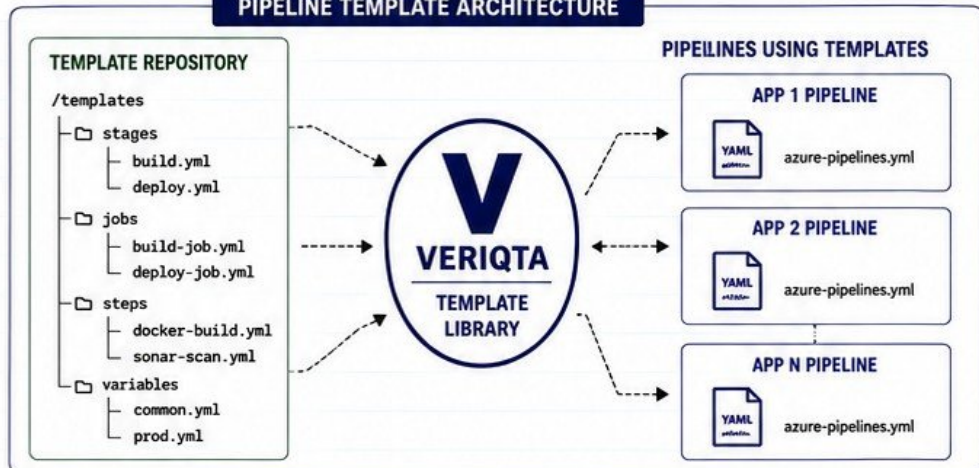
- Don't Repeat Yourself
- Build once, reuse everywhere
- Easier updates and scaling
- Reduce errors and inconsistencies



## PRODUCTION EXAMPLES

- Multi-environment deployments
- Security and compliance gates
- Reusable deployment patterns
- Enterprise-ready pipelines

## PIPELINE TEMPLATE ARCHITECTURE



## EXAMPLE TEMPLATE STRUCTURE

### STAGE TEMPLATE stages/build.yml

```
parameters:
  - name: environment
    type: string
stages:
  - stage: Build
    displayName: Build
    jobs:
      - template: ../jobs/build-job.yml
        parameters:
          environment: ${parameters.environment}
```

### JOB TEMPLATE jobs/build-job.yml

```
parameters:
  - name: environment
    type: string
jobs:
  - job: BuildJob
    displayName: Build
    steps:
      - template: ../steps/docker-build.yml
      - template: ../steps/sonar-scan.yml
```

### STEP TEMPLATE steps/docker-build.yml

```
steps:
  - task: Docker@2
    displayName: Build and Push Image
    inputs:
      command: buildAndPush
      repository: ${parameters.image}
    tags: |
      ${Build.BuildId}
```

### VARIABLE TEMPLATE variables/prod.yml

```
variables:
  - group: production
  - name: environment
    value: production
  - name: region
    value: eastus
  - name: appName
    value: myapp
```

## TEMPLATE USAGE EXAMPLE (azure-pipelines.yml)

```
1 trigger:
2   branches:
3     include:
4       - main
5
6 variables:
7   - template: variables/prod.yml@templates
8
9 stages:
10  - template: stages/build.yml@templates
11    parameters:
12      environment: ${variables.environment}
13  - template: stages/deploy.yml@templates
14    parameters:
15      environment: ${variables.environment}
```

## WHY TEMPLATES?

- ✓ Consistency across pipelines
- ✓ Faster development
- ✓ Easier maintenance
- ✓ Scalable across teams
- ✓ Better security and governance
- ✓ Enterprise standard enforcement

## ENTERPRISE TEMPLATE BEST PRACTICES



**STANDARDIZE**  
Use templates to enforce standards.



**SECURE**  
Keep sensitive data in variable groups or Key Vault.



**VERSION**  
Use semantic versioning for templates.



**REUSE**  
Build reusable components, not one-offs.



**DOCUMENT**  
Document template usage and parameters.



**GOVERN**  
Review changes, audit usage, monitor updates.

# 4. SELF-HOSTED AGENTS



## AGENT ARCHITECTURE

- Self-hosted agents execute jobs
- Connect to GitLab via HTTPS (outbound)
- Pull jobs, execute, and report status
- One agent can run multiple jobs
- Support for multiple executors



## LINUX AGENTS

- Install on Ubuntu, RHEL, or Amazon Linux
- High performance and lightweight
- Ideal for most build and deploy jobs
- Support Docker, Shell, and Kubernetes
- Easy automation with Ansible/Cloud-init



## WINDOWS AGENTS

- Install on Windows Server
- PowerShell and Batch support
- Required for .NET, MSBuild, and Windows-specific workloads
- Support Docker (Windows containers)
- Use dedicated Windows runners



## KUBERNETES AGENTS

- Run agents inside Kubernetes cluster
- Use GitLab Runner with Kubernetes executor
- Autoscale with HPA or Cluster Autoscaler
- Isolate jobs per pod for security
- Ideal for ephemeral workloads



## SCALING AGENTS

- Scale vertically: more CPU, RAM, disk
- Scale horizontally: more agent instances
- Use autoscaling groups or VM scale sets
- Use Kubernetes for dynamic scaling
- Monitor queue length and job duration



## SECURITY

- Use least privilege access
- Restrict outbound traffic
- Store tokens securely
- Isolate runners by project/group
- Keep OS and dependencies updated



## MAINTENANCE

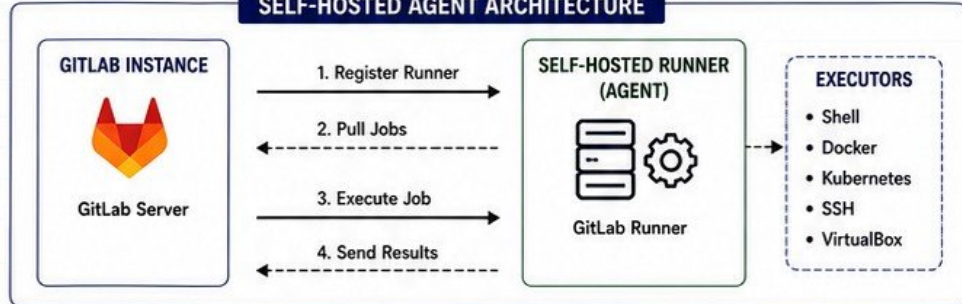
- Keep runners and dependencies updated
- Monitor system health and disk usage
- Rotate tokens periodically
- Clean up old builds and artifacts
- Backup configuration and secrets



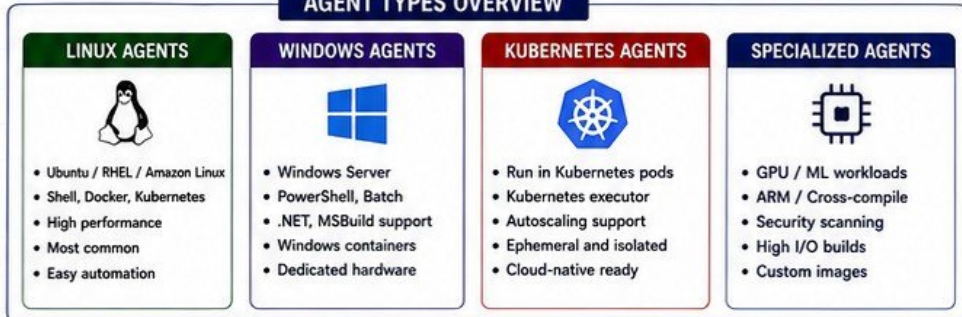
## ENTERPRISE RECOMMENDATIONS

- Use autoscaling for dynamic workloads
- Separate runners by environment
- Enforce security and compliance policies
- Monitor performance and costs
- Document and standardize everything

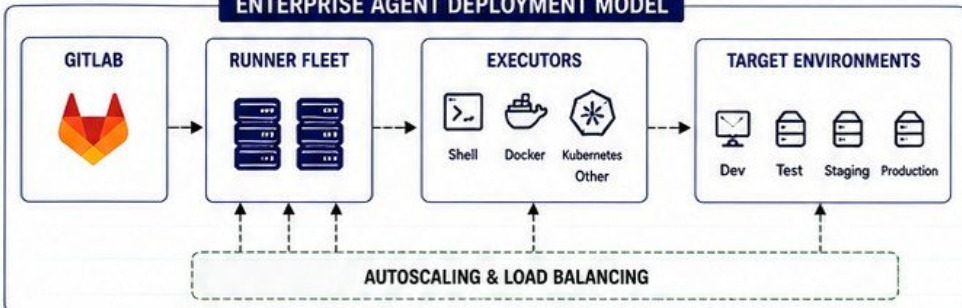
## SELF-HOSTED AGENT ARCHITECTURE



## AGENT TYPES OVERVIEW



## ENTERPRISE AGENT DEPLOYMENT MODEL



## EXAMPLE CONFIGURATION (config.toml)

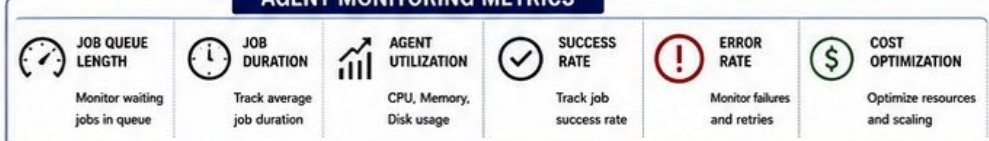
```

1 [[runners]]
2   name = "linux-docker-runner"
3   url = "https://gitlab.com/"
4   token = "YOUR_RUNNER_TOKEN"
5   executor = "docker"
6
7   [runners.docker]
8     image = "alpine:latest"
9     privileged = false
10    pull_policy = "if-not-present"
11    volumes = ["/cache"]
12
13   [runners.cache]
14     Type = "s3"
15     Path = "runner-cache"
  
```

## AGENT BEST PRACTICES

- ☒ Use autoscaling for elasticity
- ☒ Use labels to route jobs
- ☒ Separate environments
- ☒ Keep agents updated
- ☒ Monitor and alert
- ☒ Secure tokens and secrets
- ☒ Document configurations
- ☒ Test and validate changes

## AGENT MONITORING METRICS



# 5. CONTAINER-BASED PIPELINES



## DOCKER AGENTS

- Run GitLab Runner in Docker containers
- Lightweight, portable, and consistent
- Easy upgrades and version management
- Isolate runner from host environment
- Support privileged and rootless mode



## CONTAINER JOBS

- Each job runs in its own container
- Clean environment for every job
- No interference between jobs
- Use any image from any registry
- Fast startup and teardown



## BUILD CONTAINERS

- Build application Docker images
- Use Docker-in-Docker or Kaniko
- Multi-stage builds for smaller images
- Build caching for faster pipelines
- Tag, push, and promote images



## RUNTIME ISOLATION

- Strong process isolation
- Filesystem isolation
- Network isolation
- Resource limits (CPU, Memory)
- Secure and predictable execution



## CONTAINER OPTIMIZATION

- Use minimal base images (Alpine, Distroless)
- Optimize Dockerfile layers
- Leverage build cache
- Scan images for vulnerabilities
- Remove unused packages and artifacts



## REGISTRY AUTHENTICATION

- Authenticate with container registries
- Use CI variables for credentials
- Support Docker Hub, ECR, ACR, GCR
- Use Deploy Tokens or CI\_JOB\_TOKEN
- Secure access and least privilege



## PRODUCTION WORKFLOWS

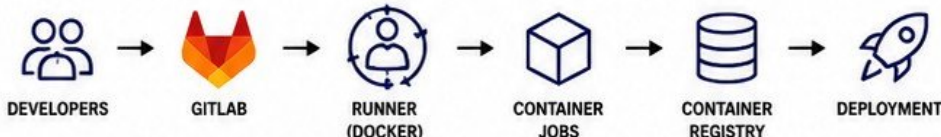
- Build → Test → Scan → Package
- Push → Deploy → Verify → Promote
- Immutable images across environments
- GitOps and environment promotion
- Rollback using image tags



## BEST PRACTICES

- Keep images small and secure
- Pin image versions (avoid latest)
- Scan images in every pipeline
- Use non-root users inside containers
- Document and standardize images

## CONTAINER-BASED PIPELINE ARCHITECTURE



## EXAMPLE .GITLAB-CI.YML (CONTAINER-BASED PIPELINE)

```

1 stages:
2   - build
3   - test
4   - scan
5   - push
6   - deploy
7
8 variables:
9   DOCKER_DRIVER: overlay2
10  DOCKER_TLS_CERTDIR: ""
11
12 image: docker:24.0.5
13 services:
14   - docker:24.0.5-dind
15
16 build:
17   stage: build
18   script:
19     - docker build -t $CI_REGISTRY_IMAGE:$CI_COMMIT_SHA .
20     - docker push $CI_REGISTRY_IMAGE:$CI_COMMIT_SHA
21
22 only:
23   - branches
  
```

## KEY BENEFITS

- ✓ Consistent environments
- ✓ Faster and repeatable builds
- ✓ Isolation and security
- ✓ Easy scaling
- ✓ Portable across platforms
- ✓ Clean and minimal agents
- ✓ Better resource utilization
- ✓ Improved reliability

## CONTAINER-BASED PIPELINE WORKFLOW



## COMMON CONTAINER IMAGES BY PURPOSE

| BUILD IMAGES                                                                                                                                       | TEST IMAGES                                                                                                                                         | UTILITY IMAGES                                                                                                                                 | SECURITY IMAGES                                                                                                                         | DEPLOY IMAGES                                                                                                                                         |
|----------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"> <li>• docker:24</li> <li>• maven:3.9-eclipse</li> <li>• node:20-alpine</li> <li>• golang:1.22-alpine</li> </ul> | <ul style="list-style-type: none"> <li>• alpine:latest</li> <li>• python:3.12-slim</li> <li>• node:20-alpine</li> <li>• gradle:8.6-jdk17</li> </ul> | <ul style="list-style-type: none"> <li>• alpine:latest</li> <li>• curlimages/curl</li> <li>• busybox:latest</li> <li>• ubuntu:24.04</li> </ul> | <ul style="list-style-type: none"> <li>• aquasec/trivy</li> <li>• snyk/snyk</li> <li>• anchore/grype</li> <li>• clair:latest</li> </ul> | <ul style="list-style-type: none"> <li>• bitnami/kubectl</li> <li>• amazon/aws-cli</li> <li>• google/cloud-sdk</li> <li>• helm/helm:latest</li> </ul> |

## CONTAINER BEST PRACTICES



### MINIMIZE IMAGE SIZE

Use minimal base images and remove unnecessary packages.



### USE MULTI-STAGE BUILDS

Build in one stage, copy artifacts to a minimal runtime image.



### SCAN IMAGES

Scan images for vulnerabilities before pushing to registry.



### RUN AS NON-ROOT

Avoid running containers as root. Use non-root users.



### PIN VERSIONS

Pin base image versions. Avoid using 'latest' tags.



### CLEAN UP

Remove unused images and containers regularly.



# 5. CONTAINER-BASED PIPELINES



## DOCKER AGENTS

- Run GitLab Runner in Docker containers
- Lightweight, portable, and consistent
- Easy upgrades and version management
- Isolate runner from host environment
- Support privileged and rootless mode



## CONTAINER JOBS

- Each job runs in its own container
- Clean environment for every job
- No interference between jobs
- Use any image from any registry
- Fast startup and teardown



## BUILD CONTAINERS

- Build application Docker images
- Use Docker-in-Docker or Kaniko
- Multi-stage builds for smaller images
- Build caching for faster pipelines
- Tag, push, and promote images



## RUNTIME ISOLATION

- Strong process isolation
- Filesystem isolation
- Network isolation
- Resource limits (CPU, Memory)
- Secure and predictable execution



## CONTAINER OPTIMIZATION

- Use minimal base images (Alpine, Distroless)
- Optimize Dockerfile layers
- Leverage build cache
- Scan images for vulnerabilities
- Remove unused packages and artifacts



## REGISTRY AUTHENTICATION

- Authenticate with container registries
- Use CI variables for credentials
- Support Docker Hub, ECR, ACR, GCR
- Use Deploy Tokens or CI\_JOB\_TOKEN
- Secure access and least privilege



## PRODUCTION WORKFLOWS

- Build → Test → Scan → Package
- Push → Deploy → Verify → Promote
- Immutable images across environments
- GitOps and environment promotion
- Rollback using image tags



## BEST PRACTICES

- Keep images small and secure
- Pin image versions (avoid latest)
- Scan images in every pipeline
- Use non-root users inside containers
- Document and standardize images

## CONTAINER-BASED PIPELINE ARCHITECTURE



## EXAMPLE .GITLAB-CI.YML (CONTAINER-BASED PIPELINE)

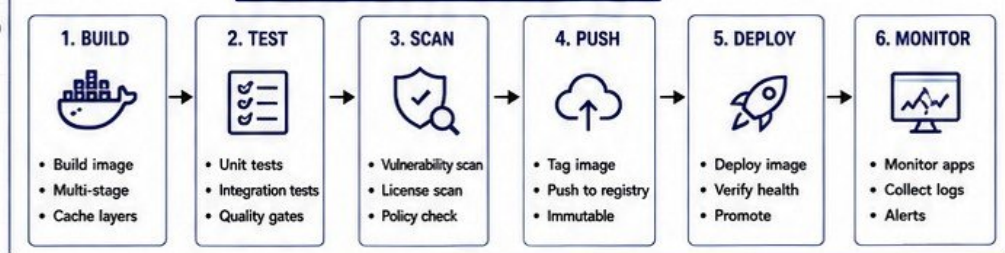
```

1 stages:
2   - build
3   - test
4   - scan
5   - push
6   - deploy
7
8 variables:
9   DOCKER_DRIVER: overlay2
10  DOCKER_TLS_CERTDIR: ""
11
12 image: docker:24.0.5
13
14 services:
15   - docker:24.0.5-dind
16
17 build:
18   stage: build
19   script:
20     - docker build -t $CI_REGISTRY_IMAGE:$CI_COMMIT_SHA .
21     - docker push $CI_REGISTRY_IMAGE:$CI_COMMIT_SHA
22 only:
23   - branches
  
```

## KEY BENEFITS

- ✓ Consistent environments
- ✓ Faster and repeatable builds
- ✓ Isolation and security
- ✓ Easy scaling
- ✓ Portable across platforms
- ✓ Clean and minimal agents
- ✓ Better resource utilization
- ✓ Improved reliability

## CONTAINER-BASED PIPELINE WORKFLOW



## COMMON CONTAINER IMAGES BY PURPOSE

### BUILD IMAGES

- docker:24
- maven:3.9-eclipse
- node:20-alpine
- golang:1.22-alpine

### TEST IMAGES

- alpine:latest
- python:3.12-slim
- node:20-alpine
- gradle:8.6-jdk17

### UTILITY IMAGES

- alpine:latest
- curlimages/curl
- busybox:latest
- ubuntu:24.04

### SECURITY IMAGES

- aquasec/trivy
- snyk/snyk
- anchore/grype
- clair:latest

### DEPLOY IMAGES

- bitnami/kubectl
- amazon/aws-cli
- google/cloud-sdk
- helm/helm:latest

## CONTAINER BEST PRACTICES



### MINIMIZE IMAGE SIZE

Use minimal base images and remove unnecessary packages.



### USE MULTI-STAGE BUILDS

Build in one stage, copy artifacts to a minimal runtime image.



### SCAN IMAGES

Scan images for vulnerabilities before pushing to registry.



### RUN AS NON-ROOT

Avoid running containers as root. Use non-root users.



### PIN VERSIONS

Pin base image versions. Avoid using 'latest' tags.



### CLEAN UP

Remove unused images and containers regularly.

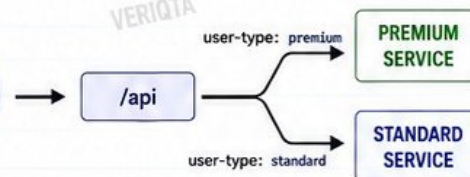
## 6. ADVANCED TRAFFIC ROUTING



### 1. HEADER-BASED ROUTING

Route traffic based on HTTP header values.

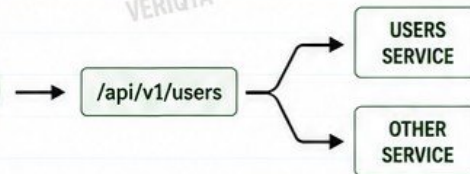
```
match:
- headers:
  user-type:
    exact: premium
```



### 2. URI ROUTING

Route traffic based on URL path patterns.

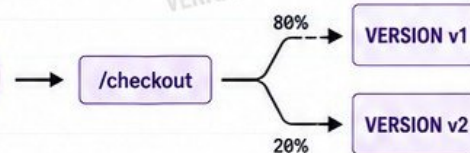
```
match:
- uri:
  prefix: /api/v1
```



### 3. WEIGHTED ROUTING

Distribute traffic across versions by weight.

```
route:
- destination: v1
  weight: 80
- destination: v2
  weight: 20
```



### 4. TRAFFIC SPLITTING

Split traffic across multiple services or versions.

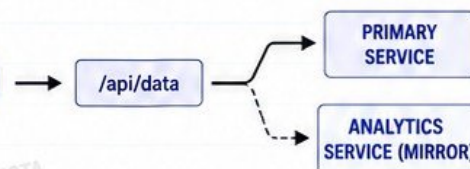
```
route:
- destination: v1
  weight: 70
- destination: v2
  weight: 30
```



### 5. TRAFFIC MIRRORING

Mirror (copy) requests to another service without affecting the response.

```
mirror:
host: analytics-svc
percent: 100
```



### 6. REDIRECTS AND REWRITES

Redirect clients or rewrite requests before routing.

```
redirect:
- uri: /old-path
  redirectCode: 301
  uri: /new-path
rewrite:
- uri: /app/(.*)
  rewrite: /$1
```

#### REDIRECT (CLIENT)

```
/old-path → 301 → /new-path
```

#### REWRITE (PROXY)

```
/app/users → rewrite → /users
```



### 7. ROUTING PRECEDENCE

Istio evaluates rules in a specific order. The first match wins.

#### ROUTING PRECEDENCE ORDER

1. Exact match (headers, URI)
2. Longer URI path match
3. Header matches
4. URI prefix match
5. Regex match
6. Catch-all (default route)

#### EXAMPLE



### EXAMPLE: COMBINED ROUTING

```
http:
- match:
  - headers:
    user-type:
      exact: premium
    uri:
      prefix: /api/v1
  route:
- destination:
  host: premium-svc
  subset: v1
- match:
  - uri:
    prefix: /api/v1
  route:
- destination:
  host: api-svc
  subset: v1
  weight: 70
- destination:
  host: api-svc
  subset: v2
  weight: 30
mirror:
host: analytics-svc
percent: 100
- match:
  - uri:
    prefix: /old
  redirect:
    uri: /new
    redirectCode: 301
- match:
  - uri:
    regex: /app/(.*)
  rewrite:
    uri: /$1
```

### BEST PRACTICES

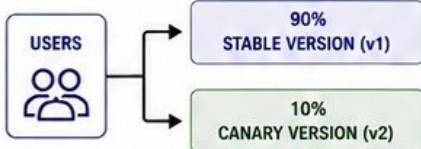
- ✓ Use header routing for A/B tests and user segmentation.
- ✓ Prefer URI routing for clean and predictable APIs.
- ✓ Use weighted routing for canary releases and gradual rollouts.
- ✓ Mirror traffic for observability without impacting users.
- ✓ Use redirects for permanent moves (301) and rewrites for clean URLs.
- ✓ Always test routing rules in lower environments before production.
- ✓ Document routing rules and maintain precedence clarity.

# 7. CANARY AND PROGRESSIVE DELIVERY



## 1. CANARY RELEASES

Release a new version to a small subset of users before rolling out to everyone.



- Reduce risk by limiting initial exposure
- Validate in real production environment
- Detect issues early



## 2. PERCENTAGE-BASED TRAFFIC

Control the percentage of traffic sent to the canary version.

### EXAMPLE TRAFFIC DISTRIBUTION

|        |         |        |
|--------|---------|--------|
| Step 1 | 5% v2   | 95% v1 |
| Step 2 | 10% v2  | 90% v1 |
| Step 3 | 25% v2  | 75% v1 |
| Step 4 | 50% v2  | 50% v1 |
| Step 5 | 100% v2 |        |



Increase gradually based on metrics and confidence.



## 3. PROGRESSIVE DELIVERY

Automatically progress traffic to the new version based on predefined criteria and metrics.

### PROGRESSIVE FLOW



- Automated or manual progression
- Metric-based decision making
- Faster delivery with safety



## 4. VERSION SUBSETS

Use version subsets to define traffic targets for routing and rollout.

### EXAMPLE VERSION SUBSETS

| Subset Name | Version | Description            |
|-------------|---------|------------------------|
| stable      | v1      | Current stable version |
| canary      | v2      | New version (canary)   |
| beta        | v2      | Beta testers           |
| internal    | v2      | Internal team only     |

- Subsets provide logical version grouping
- Simplify routing and rollout configuration



## 5. CONTROLLED ROLLOUT

Use steps, checkpoints, and approvals to control rollout speed.

### CONTROLLED ROLLOUT STAGES

|   |            |                     |
|---|------------|---------------------|
| 1 | 0% → 5%    | Initial canary      |
| 2 | 5% → 10%   | Basic validation    |
| 3 | 10% → 25%  | Extended validation |
| 4 | 25% → 50%  | Performance check   |
| 5 | 50% → 100% | Full rollout        |



Pause or stop at any stage if issues are detected.



## 6. VALIDATION

Validate the canary version using key metrics and alerts.

### WHAT TO VALIDATE

- ✓ Error rate
- ✓ Response time / latency
- ✓ Request success rate
- ✓ Resource utilization (CPU, Memory)
- ✓ Business metrics (conversion, revenue)
- ✓ Custom application metrics
- ✓ Logs and alerts



Only progress when metrics are healthy and within thresholds.



## 7. ROLLBACK STRATEGY

Have a clear and tested rollback plan to quickly recover from issues.

### ROLLBACK APPROACHES

#### INSTANT ROLLBACK



Immediately switch 100% traffic back to stable version.

#### STEP-BACK ROLLBACK



Reduce traffic to previous stable stage.

#### AUTOMATED ROLLBACK



Automatically rollback based on failed metrics/alerts.

### ROLLBACK TRIGGERS

- High error rate
- High latency
- Failed health checks
- Business metric drops
- Manual intervention



Test rollback regularly. The fastest rollback is the one you have already practiced.



Build New Version (v2)



Deploy to Canary Subset



Route Small % Traffic to v2



Monitor Metrics and Alerts



Validate Results



Increase Traffic Gradually



Full Rollout (100%)

### CANARY DELIVERY BEST PRACTICES



**START SMALL**  
Begin with a very small percentage.



**MONITOR EVERYTHING**  
Monitor technical and business metrics.



**AUTOMATE VALIDATION**  
Use automated checks and alerts.



**DOCUMENT PROCESS**  
Document rollout steps and decisions.



**COMMUNICATE**  
Keep teams informed at every stage.



**PRACTICE ROLLBACK**  
Test rollback plans regularly.

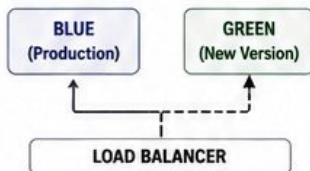
### KEY BENEFITS

- ✓ Safer releases
- ✓ Early issue detection
- ✓ Better user experience
- ✓ Data-driven decisions
- ✓ Faster and reliable delivery

# 8. DEPLOYMENT STRATEGIES

## 1. BLUE-GREEN DEPLOYMENT

Run two identical environments.  
Switch traffic from Blue to Green.



- Instant switch to new version
- Easy rollback by switching back
- Requires 2x infrastructure

### PROS

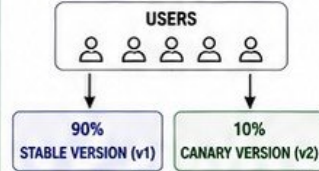
- ✓ Fast rollback
- ✓ Zero downtime
- ✓ Easy to implement

### CONS

- ✗ Higher cost
- ✗ More infrastructure

## 2. CANARY DEPLOYMENT

Deploy to a small subset of users.  
Increase traffic gradually.



- Reduce risk with small exposure
- Monitor metrics before full rollout
- Rollback by stopping traffic increase

### PROS

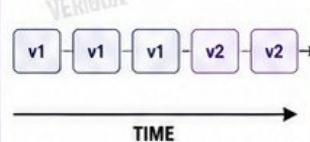
- ✓ Lower risk
- ✓ Real user validation
- ✓ Detect issues early

### CONS

- ✗ More complex
- ✗ Requires monitoring

## 3. ROLLING DEPLOYMENT

Replace old version with new version gradually.



- Update instances in batches
- Maintain availability
- Automated by orchestration tools

### PROS

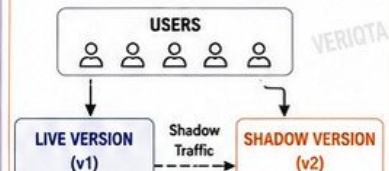
- ✓ No extra infrastructure
- ✓ Zero downtime
- ✓ Simple to automate

### CONS

- ✗ Slower rollout
- ✗ Harder rollback

## 4. SHADOW DEPLOYMENT

Send real traffic to both versions.  
Only one version serves users.



- Validate new version in production
- Compare results and performance
- No impact on user experience

### PROS

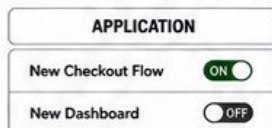
- ✓ Real traffic testing
- ✓ No user impact
- ✓ High confidence

### CONS

- ✗ Higher cost
- ✗ More complexity

## 5. FEATURE FLAGS

Release features behind flags.  
Enable for users gradually.



- Decouple deployment from release
- Enable/disable without redeploy
- Granular user or segment targeting

### PROS

- ✓ Fast control
- ✓ Safe experiments
- ✓ Granular targeting

### CONS

- ✗ Adds complexity
- ✗ Flag debt risk

## 6. PROGRESSIVE DELIVERY

Automate the delivery process.  
Use metrics to decide progression.



- Automated progression based on metrics
- Integrate with monitoring and alerts
- Pause or rollback automatically on issues

### PROS

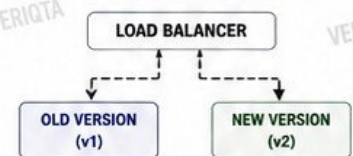
- ✓ Safer releases
- ✓ Automated decisions
- ✓ Faster delivery

### CONS

- ✗ Complex setup
- ✗ Requires good metrics

## 7. ZERO DOWNTIME DEPLOYMENT

Ensure continuous availability  
during deployment.



- Overlap old and new versions
- Health checks and connection draining
- Works with rolling, blue-green, or canary

### PROS

- ✓ No downtime
- ✓ Better user experience
- ✓ High availability

### CONS

- ✗ More complex
- ✗ Requires orchestration

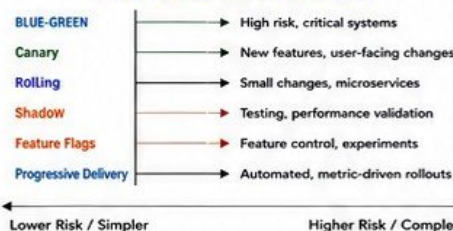
## 8. PRODUCTION EXAMPLES

| STRATEGY             | USE CASE                              | EXAMPLE SCENARIO                                           | BEST FOR                                  | TOOLS / PLATFORMS                               |
|----------------------|---------------------------------------|------------------------------------------------------------|-------------------------------------------|-------------------------------------------------|
| BLUE-GREEN           | Major upgrades, risk-free switches    | E-commerce platform deploying new checkout service         | Large systems, easy rollback requirements | AWS CodeDeploy, Azure App Service, Kubernetes   |
| CANARY               | Validate new features with real users | Roll out new search feature to 10% of users                | Web apps, user-facing features            | Istio, Argo Rollouts, Spinnaker, AWS CodeDeploy |
| ROLLING              | Routine updates, microservices        | Update API service pods one by one without downtime        | Kubernetes workloads, stateless services  | Kubernetes, Helm, Argo CD, Rancher              |
| SHADOW               | Performance testing, comparisons      | Test new recommendation engine with live traffic in shadow | ML models, performance validation         | Istio, Linkerd, AWS App Mesh, Envoy             |
| FEATURE FLAGS        | Incremental feature releases          | Enable new UI for beta users only                          | Feature control, experiments              | LaunchDarkly, Unleash, GrowthBook, ConfigCat    |
| PROGRESSIVE DELIVERY | Automated safe rollouts               | Auto increase traffic from 5% to 100% based on error rate  | DevOps teams, high reliability systems    | Argo Rollouts, Spinnaker, Flux, Istio           |

## ROLLBACK STRATEGY

- 🔍 Detect Issues: Monitor metrics, alerts, and logs.
- ⏸ Stop Progression: Pause traffic increase immediately.
- ↺ Rollback: Route traffic back to stable version.
- ✅ Validate: Ensure metrics are back to normal.
- 📄 Post-Mortem: Analyze and improve before retry.

## WHEN TO USE WHICH STRATEGY



## KEY TAKEAWAYS

- ✓ Choose the right strategy based on risk, complexity, and business impact.
- ✓ Always monitor, validate, and automate where possible.
- ✓ Start small, learn fast, and scale with confidence.
- ✓ A good deployment strategy is your best insurance in production.

# 9. INFRASTRUCTURE AS CODE



### 1. TERRAFORM INTEGRATION

Use Terraform to provision and manage infrastructure across clouds.

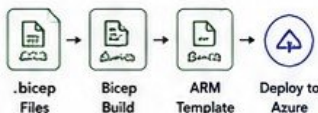


- Declarative configuration with HCL
- Plan, review, and apply changes safely
- Supports multi-cloud and hybrid



### 2. BICEP INTEGRATION

Use Bicep for concise, reusable infrastructure definitions on Azure.



- Simplified syntax for ARM
- Modules for reusability
- Built-in validation and intellisense



### 3. ARM AUTOMATION

Automate Azure infrastructure using ARM templates.



- Full control with JSON templates
- Parameterized and reusable
- Integrates with CI/CD pipelines



### 4. INFRASTRUCTURE VALIDATION

Validate templates and configurations before deployment.

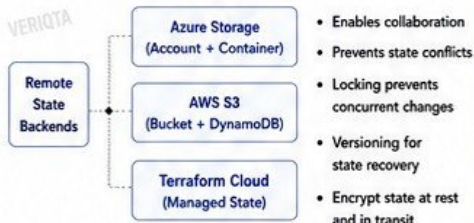


- Template syntax validation
- Policy compliance checks
- Prevent misconfigurations early



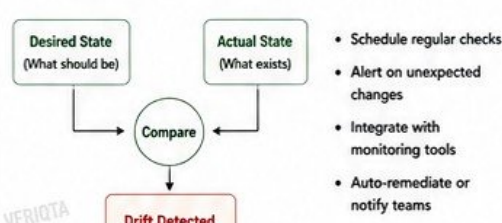
### 5. STATE MANAGEMENT

Store and manage state securely to track infrastructure changes.



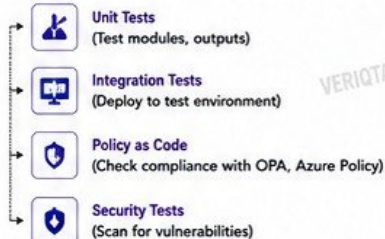
### 6. DRIFT DETECTION

Detect and identify infrastructure drift between desired and actual state.



### 7. INFRASTRUCTURE TESTING

Test infrastructure like code to ensure reliability and compliance.



### 8. ENTERPRISE AUTOMATION PIPELINE

Automate the full lifecycle of infrastructure using CI/CD.



### BEST PRACTICES

- ✓ Keep IaC code in version control
- ✓ Use modules and templates
- ✓ Review plans before apply
- ✓ Store state securely with locking
- ✓ Validate and test continuously
- ✓ Detect and remediate drift
- ✓ Follow least privilege access
- ✓ Document and tag all resources

### 9. EXAMPLE: TERRAFORM (AWS EC2)

```
provider "aws" {
  region = "us-east-1"
}

resource "aws_instance" "web" {
  ami           = "ami-0c55b159cbfafe1f0"
  instance_type = "t3.micro"
  tags = {
    Name = "web-server"
    Environment = "production"
  }
}

$ terraform init
$ terraform plan
$ terraform apply
```

### 10. EXAMPLE: BICEP (AZURE VM)

```
param location string = 'eastus'
param adminUsername string
param adminPassword string

resource vm 'Microsoft.Compute/virtualMachines@2022-08-01' = {
  name: 'web-vm'
  location: location
  properties: {
    hardwareProfile: {
      vmSize: 'Standard_B1s'
    }
    osProfile: {
      computerName: 'web-vm'
      adminUsername: adminUsername
      adminPassword: adminPassword
    }
    storageProfile: {
      imageReference: {
        publisher: 'Canonical'
        offer: '0001-com-ubuntu-server-jammy'
        sku: '22_04-lts'
        version: 'latest'
      }
    }
  }
}
```

### 11. EXAMPLE: ARM (JSON)

```
{
  "$schema": "https://schema.management.azure.com/schemas/2019-04-01/deploymentTemplate.json#",
  "contentVersion": "1.0.0.0",
  "resources": [
    {
      "type": "Microsoft.Storage/storageAccounts",
      "apiVersion": "2022-09-01",
      "name": "mystorageacct",
      "location": "[resourceGroup().location]",
      "sku": {
        "name": "Standard_LRS"
      },
      "kind": "StorageV2",
      "properties": { }
    }
  ]
}
```

### KEY BENEFITS OF IaC



## 10. AZURE KUBERNETES SERVICE PIPELINES

### 1. AKS DEPLOYMENT

Deploy and configure AKS for production workloads.



- Managed Kubernetes on Azure
- High availability with multiple nodes
- Azure CNI networking
- RBAC and Azure AD integration
- Auto-scaling node pools

```
az aks create --resource-group rg-prod \
--name aks-prod --node-count 3 \
--enable-managed-identity --generate-ssh-keys
```

### 2. KUBERNETES MANIFESTS

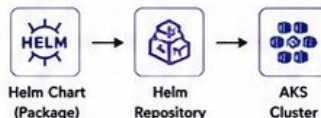
Define desired state using Kubernetes manifests.



```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web-app
spec:
  replicas: 3
  selector:
    matchLabels:
      app: web-app
  template:
    metadata:
      labels:
        app: web-app
    spec:
      containers:
        - name: web
          image: veriqa/web-app:1.0.0
          ports:
            - containerPort: 8080
```

### 3. HELM DEPLOYMENT

Package and deploy applications using Helm charts.



- Versioned and reusable charts
- Simplified upgrades and rollbacks
- Manage complex applications

```
helm repo add veriqa https://charts.veriqa.com
helm repo update
helm install web-app veriqa/web-app \
--namespace production \
--create-namespace \
--values values-prod.yaml
```

### 4. SECRETS

Store sensitive data securely.

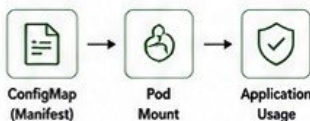


- Encrypt secrets at rest
- Use Azure Key Vault integration
- Reference secrets in pods

```
apiVersion: v1
kind: Secret
metadata:
  name: db-secret
type: Opaque
data:
  username: YWRtaW4= # base64
  password: UGFzc3dvcmQ= # base64
```

### 5. CONFIGMAPS

Manage non-sensitive configuration.



- Separate configuration from code
- Update without rebuilding images
- Support environment-specific configs

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
data:
  app.properties: |
    log.level=info
    feature.enabled=true
    api.url=https://api.prod.com
```

### 6. ROLLING UPDATES

Update applications with zero downtime.

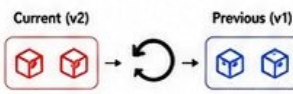


- Rolling update strategy by default
- MaxUnavailable and MaxSurge control
- Health checks ensure safe rollout

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web-app
spec:
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxUnavailable: 1
      maxSurge: 1
```

### 7. ROLLBACK

Rollback to a previous stable version quickly.



- Instant rollback to last stable revision
- Helm and kubectl rollback support
- Minimize impact and downtime

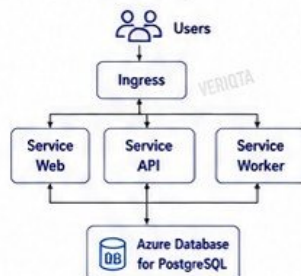
```
# Kubernetes rollback
kubectl rollout undo deployment/web-app

# Helm rollback
helm rollback web-app 1 -n production
```

### 8. PRODUCTION DEPLOYMENT

End-to-end production deployment on AKS.

- ✓ Secure and compliant
- ✓ Scalable and highly available
- ✓ Observability and monitoring
- ✓ Automated and repeatable
- ✓ Zero downtime delivery



### PRODUCTION CI/CD PIPELINE FOR AKS



Azure Repos  
GitHub

Azure Pipelines  
Maven/Gradle

Unit / Integration  
Tests

Trivy, Snyk  
Security Scan

Azure Container  
Registry (ACR)

Package Chart  
& Push

Helm Upgrade  
--install

Health Checks  
Smoke Tests

Azure Monitor  
Log Analytics

### BEST PRACTICES

- ✓ Use namespaces to isolate environments.
- ✓ Enable pod security standards.
- ✓ Use resource requests and limits.
- ✓ Store secrets in Azure Key Vault.
- ✓ Monitor with Azure Monitor and Prometheus.
- ✓ Automate everything with CI/CD.

### AKS PRODUCTION CHECKLIST

- ✓ Cluster RBAC and network policies enabled
- ✓ Managed identities for workloads
- ✓ Secrets in Key Vault, not in manifests
- ✓ Resource limits and quotas configured
- ✓ Backups and disaster recovery in place
- ✓ Monitoring, logging and alerts enabled
- ✓ Pipeline automation with approval gates
- ✓ Rollback strategy tested and documented

### TOOLS & SERVICES

- Azure AKS  
Managed Kubernetes Service
- Azure Container Registry (ACR)  
Private container registry
- Azure Key Vault  
Secrets management
- Azure Monitor & Log Analytics  
Observability and alerts
- Helm  
Package manager for Kubernetes

### KEY TAKEAWAYS

- ✓ AKS provides a reliable and scalable platform.
- ✓ Use manifests, Helm, and GitOps for repeatable deployments.
- ✓ Secure configuration with Secrets and ConfigMaps.
- ✓ Rolling updates and rollbacks ensure zero downtime.
- ✓ Automate, validate, and monitor everything.

# 11. HELM INTEGRATION

## 1. HELM PACKAGING

Package applications into reusable Helm charts.



```
helm create veriqta-app
helm lint veriqta-app
helm package veriqta-app
# Output: veriqta-app-1.0.0.tgz
```

- Standard chart structure
- Reusable and shareable
- Versioned packages

## 2. HELM UPGRADES

Upgrade releases safely with new versions and values.

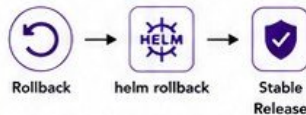


```
helm upgrade veriqta-app ./chart \
--namespace production \
--values values-prod.yaml \
--atomic --wait --timeout 10m
```

- Atomic upgrades with rollback on failure
- --wait ensures resources are ready
- --atomic rolls back if upgrade fails

## 3. HELM ROLLBACK

Rollback to a previous stable revision quickly.



```
helm rollback veriqta-app 2 \
--namespace production \
--wait --timeout 10m
# Rollback to revision 2
```

- Rollback to any previous revision
- Keeps service availability
- Fast recovery from issues

## 4. CHART VALIDATION

Validate charts before deployment to catch issues early.

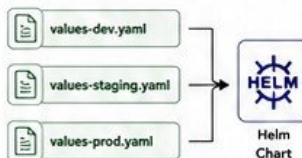


```
helm lint ./chart
helm template ./chart \
--values values-test.yaml \
--namespace test
```

- helm lint checks chart structure
- helm template renders manifests
- Prevents broken deployments

## 5. ENVIRONMENT VALUES

Use environment-specific values for different stages.



```
helm upgrade veriqta-app ./chart \
--values values-prod.yaml \
--namespace production
```

- Separate values per environment
- Override only what's needed
- Promote the same chart across envs

## 6. RELEASE MANAGEMENT

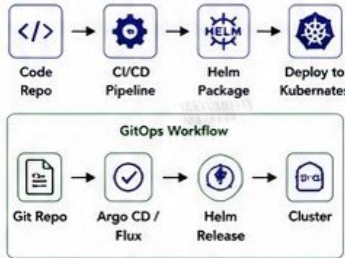
Manage Helm releases and revisions efficiently.



- Track all releases and revisions
- Audit changes with history
- Clean uninstall removes all resources

## 7. ENTERPRISE HELM WORKFLOWS

Integrate Helm into enterprise CI/CD and GitOps workflows.



- Use CI/CD for build, test, and package
- Use GitOps for continuous delivery
- Helm charts are the single source of truth

## 8. BEST PRACTICES

- ✓ Version your charts and app versions
- ✓ Use semantic versioning
- ✓ Keep charts simple and modular
- ✓ Use values files for all environments
- ✓ Encrypt sensitive data (secrets)
- ✓ Use --atomic and --wait
- ✓ Set resource requests and limits
- ✓ Use health checks and probes
- ✓ Review changes via pull requests
- ✓ Monitor releases and rollbacks
- ✓ Document chart usage and values

Consistent Helm practices lead to reliable, repeatable, and secure deployments.

## HELM CHART STRUCTURE

```
veriqta-app/
├── Chart.yaml           # Metadata
├── values.yaml          # Default values
├── values-prod.yaml     # Prod overrides
├── values-staging.yaml  # Staging overrides
├── charts/              # Dependencies
├── templates/           # Kubernetes templates
│   ├── deployment.yaml
│   ├── service.yaml
│   ├── ingress.yaml
│   └── _helpers.tpl
└── NOTES.txt
```

## COMMON HELM COMMANDS

| COMMAND                        | DESCRIPTION                 |
|--------------------------------|-----------------------------|
| helm create <name>             | Create a new chart          |
| helm package <chart>           | Package chart into .tgz     |
| helm repo add <name> <url>     | Add Helm repository         |
| helm repo update               | Update repo index           |
| helm install <release> <chart> | Install a new release       |
| helm upgrade <release> <chart> | Upgrade an existing release |
| helm rollback <release> <rev>  | Rollback to a revision      |
| helm uninstall <release>       | Uninstall a release         |
| helm list -A                   | List all releases           |
| helm status <release>          | Show release status         |

## EXAMPLE: PRODUCTION UPGRADE FLOW

- 1 → Develop & Test  
Code changes, test locally
- 2 → Package Chart  
helm package ./chart
- 3 → Push to Repo  
Publish chart to artifact repo
- 4 → Upgrade in CI/CD  
helm upgrade with prod values
- 5 → Verify & Monitor  
Validate and monitor health

## HELM VALUES EXAMPLE (values-prod.yaml)

```
replicaCount: 3
image:
  repository: veriqta/web-app
  tag: 1.2.0
  pullPolicy: IfNotPresent
service:
  type: ClusterIP
  port: 80
ingress:
  enabled: true
  hosts:
    - host: app.veriqta.com
      path: /
      pathType: Prefix
resources:
  requests:
    cpu: "200m"
    memory: "256Mi"
  limits:
    cpu: "500m"
    memory: "512Mi"
```

## HELM TEMPLATE OUTPUT (helm template ./chart --values values-prod.yaml)

```
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: veriqta-app
  labels:
    app: veriqta-app
spec:
  replicas: 3
  selector:
    matchLabels:
      app: veriqta-app
  template:
    metadata:
      labels:
        app: veriqta-app
    spec:
      containers:
        - name: web
          image: veriqta/web-app:1.2.0
          ports:
            - containerPort: 80
          resources:
            requests:
              cpu: 200m
              memory: 256Mi
            limits:
              cpu: 500m
              memory: 512Mi
```

## RELEASE REVISION HISTORY EXAMPLE

| REVISION | UPDATED          | STATUS     | CHART             | APP VERSION |
|----------|------------------|------------|-------------------|-------------|
| 5        | 2025-05-16 12:30 | deployed   | veriqta-app-1.2.0 | 1.2.0       |
| 4        | 2025-05-15 10:20 | superseded | veriqta-app-1.1.0 | 1.1.0       |
| 3        | 2025-05-14 09:10 | superseded | veriqta-app-1.0.1 | 1.0.1       |
| 2        | 2025-05-12 16:45 | superseded | veriqta-app-1.0.0 | 1.0.0       |
| 1        | 2025-05-10 11:05 | deployed   | veriqta-app-1.0.0 | 1.0.0       |

✓ TIP: Always test upgrades in lower environments before promoting to production.

# 12. GITOPS WITH AZURE DEVOPS



### GITOPS PRINCIPLES

- Git is the single source of truth
- Declarative and version controlled
- Automated operations
- Continuous reconciliation
- Immutable infrastructure
- Auditability and traceability



### REPOSITORY STRUCTURE

```
gitops-repo/
├── clusters/
│   ├── dev/
│   │   ├── kustomization.yaml
│   │   └── namespace.yaml
│   ├── staging/
│   └── prod/
├── apps/
│   ├── web-app/
│   │   ├── kustomization.yaml
│   │   ├── deployment.yaml
│   │   └── service.yaml
│   └── api-app/
├── base/
│   ├── deployment.yaml
│   ├── service.yaml
│   └── configmap.yaml
├── overlays/
│   ├── dev/
│   ├── staging/
│   └── prod/
└── README.md
```



### DESIRED STATE

- All manifests in Git define the desired state
- Changes are made by updating Git
- Systems automatically converge to desired state
- No manual changes in target environment



### GIT WORKFLOWS

- Feature branch workflow
- Pull request for changes
- Code review and approvals
- Merge to main for deployment
- Environments mapped to branches or paths



### CONTINUOUS RECONCILIATION

- Controllers constantly compare Git state with cluster state
- Drift is detected and corrected automatically
- Self-healing infrastructure
- Real-time or periodic sync



### ENTERPRISE DEPLOYMENT

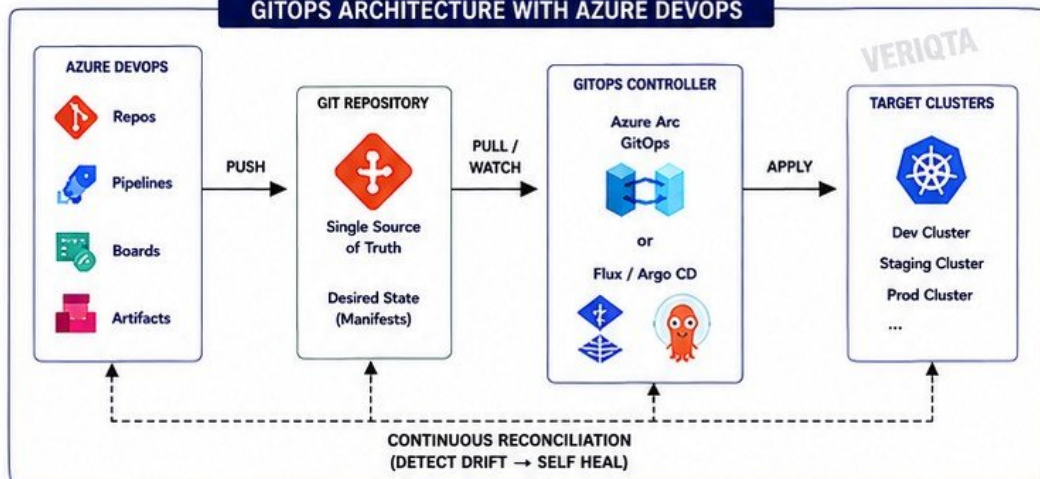
- Multi-cluster management
- Environment isolation
- Policy and compliance gates
- Secrets management
- RBAC and least privilege
- Monitoring and alerts



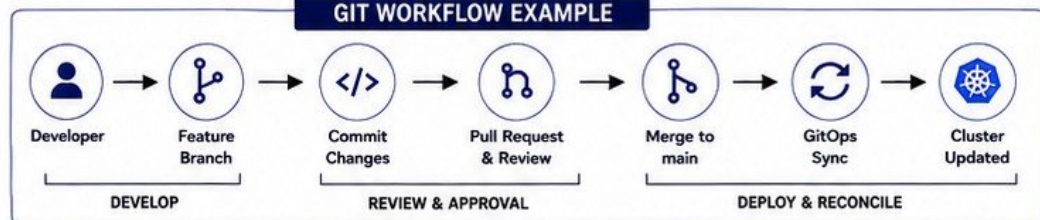
### BEST PRACTICES

- Keep configurations modular
- Use meaningful commit messages
- Protect main branch
- Automate validation and testing
- Monitor reconciliation and drift
- Document everything

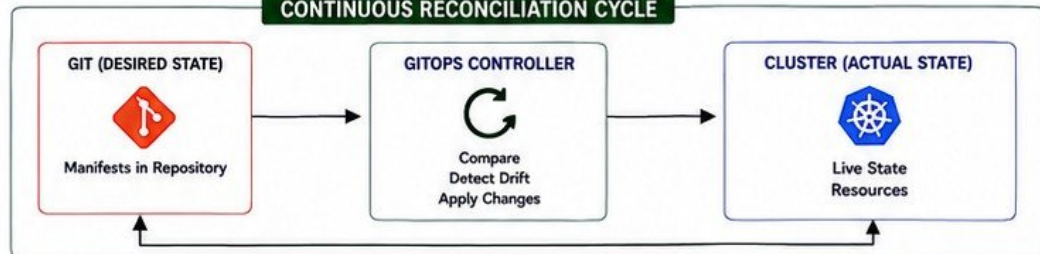
### GITOPS ARCHITECTURE WITH AZURE DEVOPS



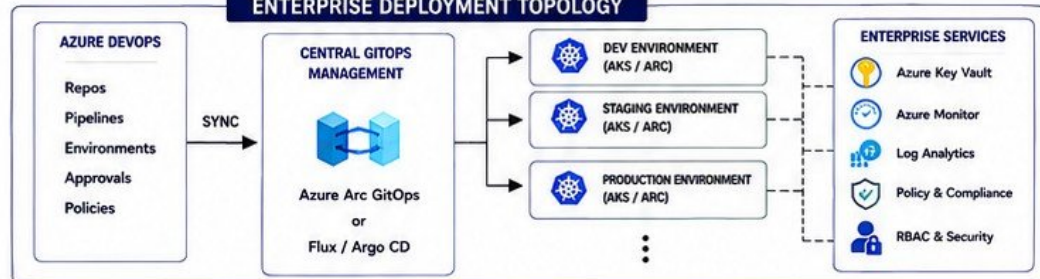
### GIT WORKFLOW EXAMPLE



### CONTINUOUS RECONCILIATION CYCLE



### ENTERPRISE DEPLOYMENT TOPOLOGY



### GITOPS BEST PRACTICES



# ARGO CD INTEGRATION

**GITOPS ENGINE** 



## ① ARGO CD OVERVIEW

- Declarative, GitOps continuous delivery tool
- Syncs Kubernetes manifests from Git to cluster
- Detects drift and automatically corrects
- Supports multi-cluster and multi-tenant
- UI, CLI and declarative APIs



## ② AZURE DEVOPS INTEGRATION

- Store Kubernetes manifests in Azure Repos
- Trigger Argo CD sync after successful build
- Use Service Connection or PAT for access
- Webhook integration for automated sync
- End-to-end CI/CD + GitOps pipeline



## ③ SYNC STRATEGIES

- Manual Sync – on demand
- Automatic Sync – real-time
- Prune – remove deleted resources
- Self Heal – fix drift automatically
- Sync Waves – control resource order



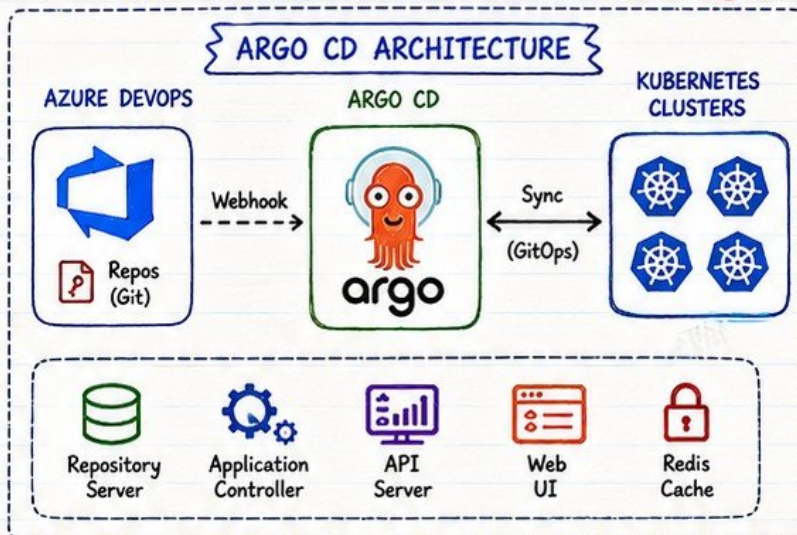
## ④ DRIFT DETECTION

- Continuously compares live state with Git
- Highlights OutOfSync resources
- Shows differences in UI and CLI
- Auto-correct with Self Heal
- Prevents configuration drift



## ⑤ ROLLBACKS

- Rollback to previous Git commit
- Rollback to previous Argo CD revision
- UI, CLI or declarative rollback
- Automated rollback on health failure
- Keep history for traceability



## ⑥ MULTI-CLUSTER DEPLOYMENTS

- Manage multiple clusters from one Argo CD
- Add clusters using kubeconfig or Service Account
- Cluster labels for environment separation
- Projects for RBAC and isolation
- Centralized GitOps for large environments



## ⑦ GITOPS AUTOMATION

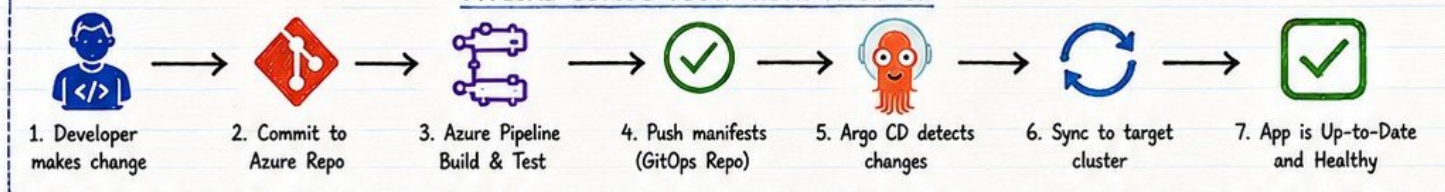
- Commit changes to Git
- Argo CD detects and syncs automatically
- Policies ensure consistency and compliance
- Integrate with CI pipeline for validation
- Complete automation from code to production



## ⑧ PRODUCTION EXAMPLES

- Microservices app on AKS with auto sync
- Multi-environment promotion (dev → prod)
- Canary deployment with Argo Rollouts
- DR cluster sync across regions
- GitOps for platform & infra components

### TYPICAL GITOPS FLOW WITH ARGO CD



 **TIP:** Always use Automatic Sync + Prune + Self Heal for true GitOps automation. 

# VERIQTA PRODUCTION HANDBOOK

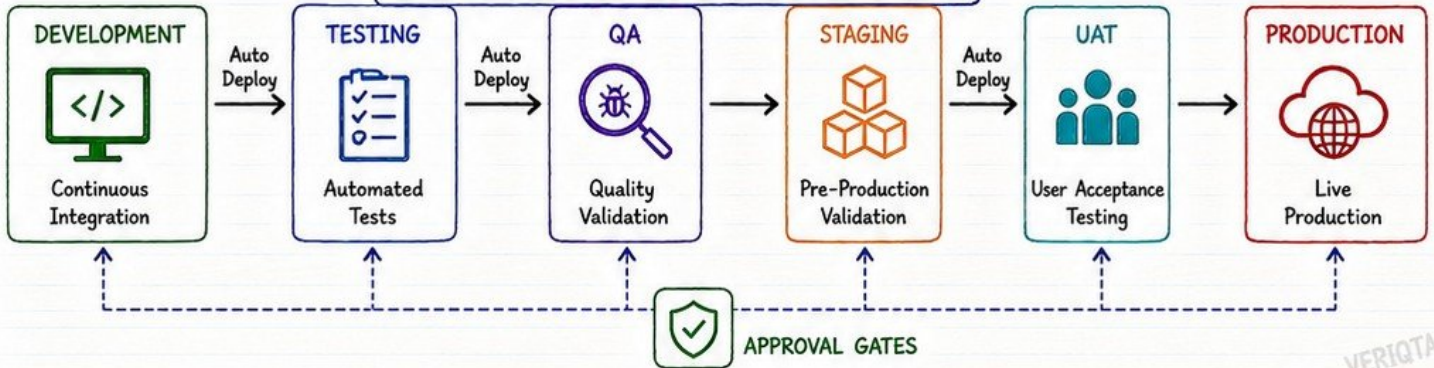
## MULTI-ENVIRONMENT DEPLOYMENTS

PAGE  
**14**

GITOPS ENGINE

Deliver applications safely and consistently across all environments with automated promotion, approvals and governance.

### MULTI-ENVIRONMENT PIPELINE FLOW



### ENVIRONMENT PURPOSE

|             |                                                                 |
|-------------|-----------------------------------------------------------------|
| DEVELOPMENT | Develop features, unit tests, and continuous integration.       |
| TESTING     | Automated functional, integration and regression testing.       |
| QA          | Manual testing, exploratory testing and defect validation.      |
| STAGING     | Mirror production, performance and final validation.            |
| UAT         | Business users validate the application before go-live.         |
| PRODUCTION  | Live environment for end users with full monitoring and alerts. |

### PROMOTION STRATEGY

- Code Commit**  
Developers commit code to main branch.
- Build & Test**  
Azure DevOps builds, runs tests and publishes artifacts.
- Deploy to Dev**  
Automatic deployment to Development environment.
- Validate & Approve**  
Quality checks and approvals at each stage.
- Promote Upwards**  
Promote artifacts/configurations to next environment.
- Deploy to Production**  
Final approval and deploy to Production.

### ENVIRONMENT GOVERNANCE

- ACCESS CONTROL**
  - RBAC for environments
  - Least privilege access
- APPROVALS & GATES**
  - Mandatory approvals
  - Quality and security gates
- POLICIES & STANDARDS**
  - Deployment standards
  - Naming, tagging, and conventions
- AUDIT & COMPLIANCE**
  - Track all deployments
  - Audit logs and reports
- MONITORING & ALERTS**
  - Environment health monitoring
  - Alerts and notifications
- CONFIGURATION MANAGEMENT**
  - Infrastructure as Code
  - Versioned and traceable configs

### ENVIRONMENT CONFIGURATION EXAMPLE

|                      |                                   |
|----------------------|-----------------------------------|
| Environment Name     | dev, test, qa, staging, uat, prod |
| Azure Subscription   | Separate per environment          |
| Resource Group       | rg-app-dev, rg-app-test, ...      |
| Kubernetes Namespace | app-dev, app-test, ...            |
| App Configuration    | Environment-specific values       |
| Secrets              | Stored in Key Vault per env       |
| Monitoring           | Environment-specific dashboards   |
| Backup Strategy      | Based on environment criticality  |

### BEST PRACTICES

- ✓ Use infrastructure as code for all environments.
- ✓ Automate deployments and validations.
- ✓ Promote the same artifact, never rebuild.
- ✓ Keep environments as similar as possible.
- ✓ Enforce approvals for critical environments.
- ✓ Monitor, measure and continuously improve.

### TIP

Automate everything you can, but keep human approval where risk is high. Balance speed with safety.

# ENTERPRISE SECRETS MANAGEMENT

SECURE BY  
DESIGN



GITOPS ENGINE



Secure, automate and govern secrets across the enterprise with Azure Key Vault and modern DevOps practices.

## ① AZURE KEY VAULT



- Centralized secret store in Azure.
- Store secrets, keys and certificates.
- HSM-backed for enterprise security.
- RBAC + Access policies.

## ② SECRET ROTATION



- Automate secret rotation.
- Reduce the risk of compromise.
- Rotate keys, passwords, tokens and certificates.
- Use Event Grid + Functions / Logic Apps.

## ③ CERTIFICATES



- Store TLS/SSL certificates securely.
- Auto-renew certificates with Key Vault Certificate Manager.
- Support for PFX, PEM and CER formats.

## ④ MANAGED IDENTITIES



- Eliminate static credentials.
- Use System Assigned or User Assigned Managed Identities.
- Secure access to Key Vault and Azure resources.

## ⑤ SECURE PIPELINES



- Integrate Key Vault in Azure DevOps pipelines.
- Use service connections with Managed Identity.
- Fetch secrets at runtime, not store.
- Never hardcode secrets in YAML or scripts.

## ⑥ SECURE VARIABLES



- Use variable groups linked to Key Vault.
- Mark variables as secret.
- Limit access to variable groups.
- Audit variable usage.

## ⑦ COMPLIANCE



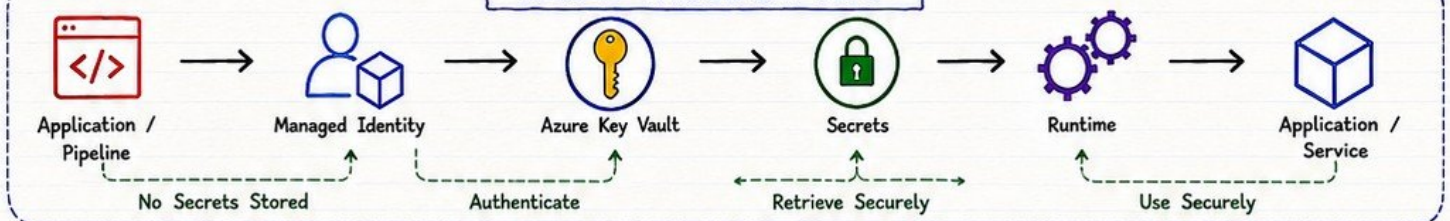
- Meet enterprise compliance needs.
- Audit access with Key Vault logs.
- Support for ISO 27001, SOC 2, PCI DSS, HIPAA and more.

## ⑧ PRODUCTION SECURITY



- Least privilege access.
- Network isolation with Private Endpoint.
- Enable soft delete and purge protection.
- Monitor, detect and respond to threats.

### SECRETS MANAGEMENT FLOW



### YAML EXAMPLE: USE KEY VAULT IN AZURE DEVOPS

```

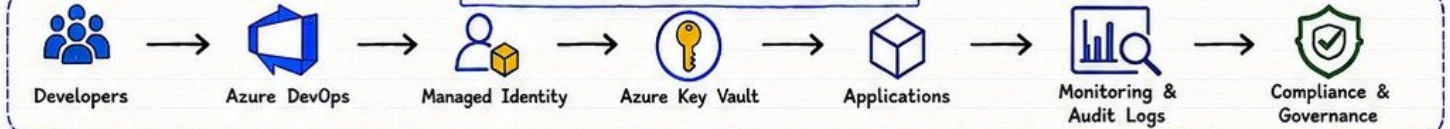
1 variables:
2   - group: kv-variables
3 steps:
4   - task: AzureKeyVault@2
5     inputs:
6       azureSubscription: 'My-Service-Connection'
7       KeyVaultName: 'veriqta-kv'
8       SecretsFilter: 'dbPassword,apiKey'
9   - script: |
10     echo "Using secret securely at runtime"
11     ./deploy.sh
  
```

Secrets are retrieved at runtime and never stored in code.

### BEST PRACTICES

- ✓ Use Managed Identities - never store credentials.
- ✓ Store all secrets in Azure Key Vault.
- ✓ Rotate secrets and keys regularly.
- ✓ Enable soft delete and purge protection.
- ✓ Restrict network access with Private Endpoint.
- ✓ Audit access and monitor with Azure Monitor.
- ✓ Use JIT (Just-In-Time) access where possible.
- ✓ Follow least privilege access principle.
- ✓ Tag and classify secrets based on sensitivity.

### ENTERPRISE SECRETS ARCHITECTURE



# ENTERPRISE SECURITY

SECURE BY DESIGN 

GITOPS ENGINE 

Secure your Azure DevOps environment end-to-end with policies, controls and best practices.

## 1 RBAC



- Enforce least privilege access.
- Use groups and custom roles.
- Scope permissions at project level.
- Review and audit access regularly.

## 2 CONDITIONAL ACCESS



- Require MFA for all users.
- Restrict access by location, device and network.
- Use sign-in risk policies.
- Block legacy authentication.

## 3 BRANCH PROTECTION



- Protect main and release branches.
- Require PR reviews and status checks.
- Enforce build validation policies.
- Prevent force push and deletion.

## 4 PIPELINE SECURITY



- Approve pipeline changes.
- Restrict who can run pipelines.
- Use environment approvals.
- Scan and validate before deployment.

## 5 SECURE REPOSITORIES



- Restrict repo access to authorized users.
- Enable repository auditing.
- Encrypt sensitive data in repos.
- Remove secrets from code.

## 6 SECURE AGENTS



- Use Microsoft-hosted or trusted agents.
- Keep agents updated and hardened.
- Limit network access from agents.
- Clean up agents regularly.

## 7 SUPPLY CHAIN SECURITY



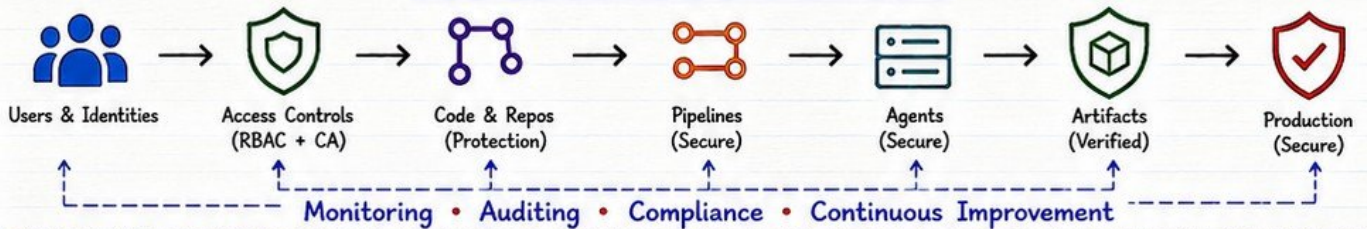
- Verify dependencies and third-party components.
- Use signed commits and tags.
- Scan artifacts and container images.
- Implement SLSA best practices.

## 8 ENTERPRISE RECOMMENDATIONS



- Establish security as a shared responsibility.
- Automate security checks and policies.
- Monitor, alert and respond quickly.
- Continuously improve security posture.

### ENTERPRISE SECURITY CONTROL FLOW



### SECURITY CHECKLIST

- ☒ MFA enabled for all users
- ☒ RBAC enforced and reviewed
- ☒ Branch protection enforced
- ☒ Pipelines require approvals and checks
- ☒ Secrets stored in Key Vault, not in code
- ☒ Agents are secure, updated and restricted
- ☒ Dependency and artifact scanning enabled
- ☒ Audit logs enabled and monitored



### SECURITY PRINCIPLES

- ✓ **Least Privilege** - Give only the access required.
- ✓ **Defense in Depth** - Multiple layers of protection.
- ✓ **Secure by Design** - Build security into everything.
- ✓ **Automate Security** - Use automation, not manual.
- ✓ **Visibility & Monitoring** - You can't protect what you can't see.
- ✓ **Continuous Improvement** - Evolve and adapt to new threats.





**VERIQTA**  
• ENGINEERED FOR IMPACT •

# DEVSECOPS INTEGRATION

SECURE BY  
DESIGN

GITOPS ENGINE

Shift security left, automate security everywhere  
and build secure software at enterprise scale.

## ① SAST



- Static Application Security Testing.
- Scan source code for vulnerabilities.
- Detects issues early in SDLC.
- Tools: SonarQube, Checkmarx, Semgrep.

## ② DAST



- Dynamic Application Security Testing.
- Test running applications.
- Find runtime vulnerabilities.
- Tools: OWASP ZAP, Burp Suite.

## ③ DEPENDENCY SCANNING



- Scan open source and third-party dependencies.
- Detect vulnerable libraries.
- Tools: Snyk, OWASP Dependency Check, GitHub Dependabot.

## ④ CONTAINER SCANNING



- Scan container images for vulnerabilities.
- Base image and OS package scanning.
- Tools: Trivy, Clair, Anchore.

## ⑤ SECRET SCANNING



- Detect secrets in code, configs and commits.
- Prevent credential leaks.
- Tools: GitGuardian, TruffleHog, GitLeaks.

## ⑥ SECURITY GATES



- Automated checks that must pass before promotion.
- Fail builds on high/critical issues.
- Enforce policy as code.

## ⑦ POLICY ENFORCEMENT



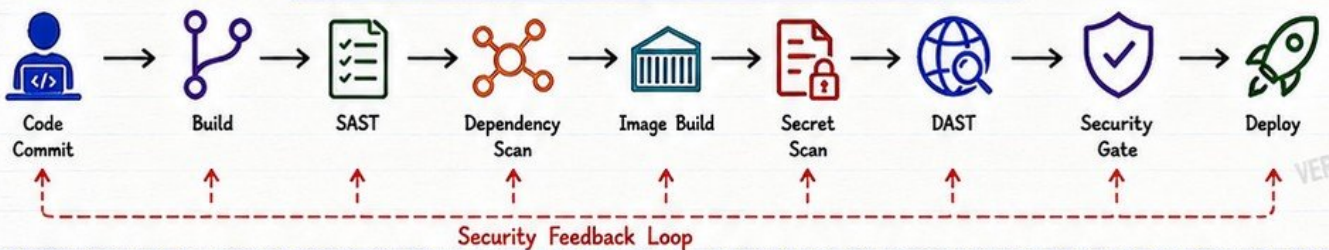
- Enforce security policies across pipelines.
- Use OPA, Kyverno or Azure Policy.
- Ensure compliant infrastructure and workloads.

## ⑧ ENTERPRISE DEVSECOPS



- Security embedded in culture.
- Shared responsibility across teams.
- Continuous visibility and improvement.
- Measure, monitor, and mature.

### DEVSECOPS PIPELINE FLOW (SHIFT LEFT + AUTOMATE)



### SECURITY TOOLS MAPPING

| SECURITY AREA       | EXAMPLE TOOLS                                   |
|---------------------|-------------------------------------------------|
| SAST                | SonarQube, Checkmarx, Semgrep                   |
| DAST                | OWASP ZAP, Burp Suite, Netsparker               |
| DEPENDENCY SCANNING | Snyk, OWASP Dependency Check, GitHub Dependabot |
| CONTAINER SCANNING  | Trivy, Clair, Anchore, Aqua Security            |
| SECRET SCANNING     | GitLeaks, TruffleHog, GitGuardian               |
| POLICY ENFORCEMENT  | OPA, Kyverno, Azure Policy                      |
| COMPLIANCE & AUDIT  | Azure Defender, Prisma Cloud, Lacework, Splunk  |

### BEST PRACTICES

- ✓ Perform security early and continuously.
- ✓ Automate all security scans and checks.
- ✓ Fail fast on high and critical issues.
- ✓ Keep dependencies and images up to date.
- ✓ Scan everything: code, config, images, secrets.
- ✓ Use policy as code for consistent enforcement.
- ✓ Centralize security dashboards and alerts.
- ✓ Train teams and build security culture.
- ✓ Measure, monitor and continuously improve.



### TIP

Security is not a checkpoint, it is a continuous practice. Build it in, automate it, and make it everyone's responsibility.

**VERIQTA**



YouTube  
/veriqta



GitHub  
/veriqta



LinkedIn  
/company/veriqta



X  
@veriqta



Instagram  
@veriqta\_academy



Telegram  
t.me/veriqta



ENGINEERED FOR IMPACT



BUILT FOR SCALE



AUTOMATED FOR EXCELLENCE





VERIQTA

• ENGINEERED FOR IMPACT •

## VERIQTA PRODUCTION HANDBOOK

# POLICY AS CODE

PAGE  
18

SECURE BY  
DESIGN



GITOPS ENGINE



Define, enforce and automate governance and compliance across your Azure environments using Policy as Code.

### 1 AZURE POLICY



- Enforce rules and standards for Azure resources.
- Built-in and custom policies.
- Evaluate resources at scale.

### 2 COMPLIANCE AUTOMATION



- Continuously assess compliance.
- Auto-remediate non-compliant resources.
- Generate compliance reports.

### 3 GOVERNANCE



- Centralized governance across subscriptions.
- Initiative definitions for policy grouping.
- Role-based access for governance.

### 4 PIPELINE POLICIES



- Validate infrastructure and app changes.
- Prevent non-compliant deployments.
- Integrate with Azure DevOps pipelines.

### 5 ENVIRONMENT POLICIES



- Different policies for dev, test, prod.
- Enforce environment-specific standards.
- Tag and location restrictions.

### 6 AUDIT AUTOMATION



- Log policy evaluations and changes.
- Alert on violations and exceptions.
- Integrate with Log Analytics and Azure Monitor.

### 7 ENTERPRISE GOVERNANCE



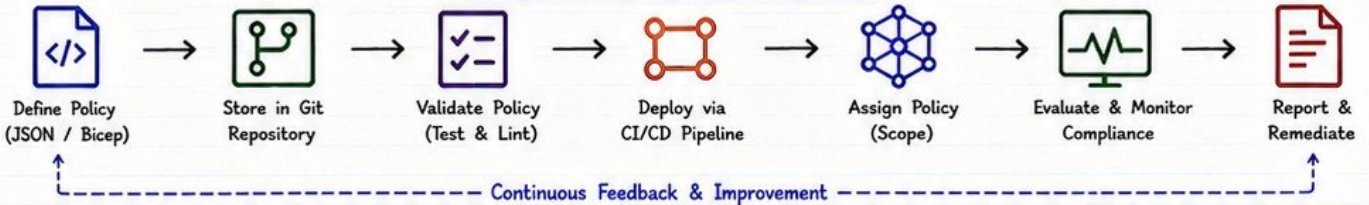
- Organizational ownership and accountability.
- Change management for policies.
- Governance at scale with consistency.

### 8 BEST PRACTICES



- Start with audit, then enforce.
- Version control all policies as code.
- Test policies before assigning.
- Continuously review and improve.

#### POLICY AS CODE WORKFLOW



#### AZURE POLICY STRUCTURE (EXAMPLE)

```
1 {
2   "properties": {
3     "displayName": "Restrict Public IP",
4     "policyType": "Custom",
5     "mode": "All",
6     "policyRule": {
7       "if": {
8         "field": "type",
9         "equals": "Microsoft.Network/publicIPAddresses"
10      },
11      "then": {
12        "effect": "Deny"
13      }
14    }
15  }
16 }
```

Policies are defined in JSON or Bicep and versioned in source control.

#### COMMON POLICY CATEGORIES



**SECURITY**  
Enforce security baselines and prevent risky configurations.



**NETWORKING**  
Control network access, NSG rules, and endpoints.



**COST MANAGEMENT**  
Enforce tagging, region, and resource quotas.



**COMPLIANCE**  
Meet industry standards and regulatory requirements.



**IDENTITY**  
Enforce RBAC, MFA and identity policies.



**STORAGE**  
Enforce encryption, replication and access policies.



**MONITORING**  
Ensure diagnostics and monitoring are enabled.

#### POLICY ENFORCEMENT MODES



**AUDIT**  
Monitor and log non-compliance without blocking.



**ENFORCE**  
Block or deny non-compliant deployments.

#### SCOPE HIERARCHY



Management Group



Subscription



Resource Group



Resource

#### KEY BENEFITS

- ✓ Consistent governance across all environments.
- ✓ Reduce risk and enforce security by default.
- ✓ Automate compliance and reduce manual work.
- ✓ Improve visibility, control and accountability.

VERIQTA



YouTube  
/veriqta



GitHub  
/veriqta



LinkedIn  
/company/veriqta



X  
@veriqta



Instagram  
@veriqta\_academy



Telegram  
t.me/veriqta



ENGINEERED FOR IMPACT • BUILT FOR SCALE • AUTOMATED FOR EXCELLENCE



Gain complete visibility into your Azure environments, applications and services for reliable production operations.

## 1 AZURE MONITOR



- Centralized monitoring for all Azure resources.
- Collect metrics, logs and activity.
- Built-in workbooks and solutions.
- End-to-end visibility across subscriptions.

## 2 APPLICATION INSIGHTS



- Monitor application performance and health.
- Detect exceptions, failures and slow requests.
- Track user behavior and dependencies.
- Supports .NET, Java, Node.js, Python and more.

## 3 LOG ANALYTICS



- Collect and analyze logs at scale.
- Powerful KQL queries for deep insights.
- Correlate data from multiple sources.
- Retain, search and export logs securely.

## 4 DASHBOARDS



- Create custom dashboards.
- Visualize metrics, logs and KPIs.
- Share and embed dashboards.
- Real-time insights at a glance.

## 5 METRICS



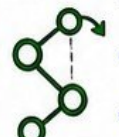
- Monitor platform and custom metrics.
- High cardinality metric support.
- Aggregate and visualize trends.
- Monitor SLIs, SLOs and resource usage.

## 6 ALERTS



- Create metric, log and activity alerts.
- Multi-level severity and conditions.
- Action groups: email, SMS, webhook, Logic Apps and more.
- Reduce MTTR with proactive alerts.

## 7 DISTRIBUTED TRACING



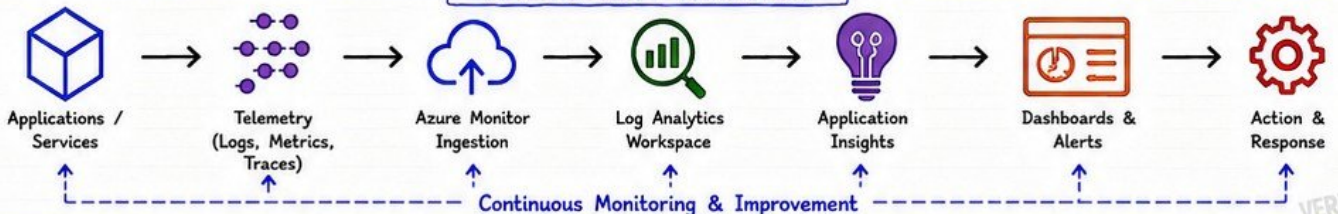
- Trace requests across services.
- Identify latency bottlenecks.
- End-to-end request correlation.
- Works with App Insights and OpenTelemetry.

## 8 PRODUCTION MONITORING



- 24/7 monitoring of production workloads.
- Detect anomalies and incidents.
- Ensure reliability, performance and availability.
- Drive operational excellence.

### OBSERVABILITY DATA FLOW



### SAMPLE KQL QUERY (LOG ANALYTICS)

```

requests
| where timestamp > ago(1h)
| where success == false
| summarize count() by resultCode
| order by count_desc
  
```

Use KQL to search, filter and analyze logs at scale.

### USEFUL METRICS TO TRACK

- |                     |                                 |
|---------------------|---------------------------------|
| ✓ CPU Percentage    | ✓ Response Time (P50, P95, P99) |
| ✓ Memory Usage      | ✓ Exceptions                    |
| ✓ Disk Read / Write | ✓ Dependency Duration           |
| ✓ Requests / Second | ✓ Queue Length                  |
| ✓ Failed Requests   | ✓ Availability %                |

### PRODUCTION MONITORING STACK

#### DATA SOURCES

- Azure Resources
- Applications
- Containers
- Network & Security
- 3rd Party Systems

#### COLLECTION

- Azure Monitor Agent
- Application Insights SDK
- Diagnostic Settings
- Event Hubs / Log Ingestion

#### ANALYSIS & STORAGE

- Log Analytics Workspace
- Data Retention & Archive
- Workbooks & Queries

#### VISUALIZATION & ALERTING

- Dashboards
- Alerts
- Action Groups
- On-Call Notification

#### ACTION & RESPONSE

- Ops / Dev Teams
- Automation
- Incident Resolution
- Continuous Improvement



**VERIQTA**  
• ENGINEERED FOR IMPACT •

# PRODUCTION TROUBLESHOOTING

SECURE BY  
DESIGN

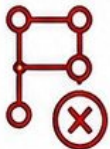


GITOPS ENGINE



Diagnose, analyze and recover from production issues  
fast – with clarity, structure and proven workflows.

## 1 PIPELINE FAILURES



- Build failures, test failures, or job errors.
- Broken tasks or scripts.
- Artifact publish/download issues.
- Approval or gate rejections.
- Misconfigured variables or service connections.

## 2 DEPLOYMENT FAILURES



- Failed releases or rollback loops.
- Health check failures.
- Misconfigurations in manifests or templates.
- Image pull errors.
- Service or app startup failures.

## 3 AGENT FAILURES



- Agent offline or unreachable.
- Capability or version mismatch.
- Job stuck or queued forever.
- Permission issues on agent.
- Resource exhaustion on agent machine.

## 4 INFRASTRUCTURE FAILURES



- VM/VMSS/Scale Set unavailable.
- Network, DNS, or connectivity issues.
- Storage or disk failures.
- Azure service outages or throttling.
- Quota or limit exceeded.

## 5 KUBERNETES FAILURES



- Pod crash loop or not ready.
- Image pull back-off.
- Pending pods (scheduling issues).
- Service, Ingress or DNS failures.
- Resource limits, OOMKilled, Evictions.
- Node Not Ready or network issues.

## 6 ROOT CAUSE ANALYSIS



- Gather facts and preserve evidence.
- Identify the 5 Whys and failure chain.
- Correlate metrics, logs and events.
- Isolate blast radius and impact.
- Determine primary cause vs symptoms.

## 7 RECOVERY WORKFLOWS



- Contain the issue and stop the impact.
- Restore service (rollback or hotfix).
- Validate health and performance.
- Communicate status and resolution.
- Post-mortem and improvement plan.

## 8 SENIOR ENGINEER DEBUGGING



- Think in systems and dependencies.
- Use data, not assumptions.
- Reproduce and simplify.
- Verify, then fix.
- Automate to prevent recurrence.

### PRODUCTION INCIDENT RESPONSE WORKFLOW



**Detect**  
Alert, user report, or monitoring.



**Triage**  
Assess severity, impact, urgency.



**Analyze**  
Collect data, logs, metrics, events.



**Identify Root Cause**  
Validate facts, isolate issue.



**Fix / Recover**  
Implement fix or rollback.



**Validate**  
Confirm service is healthy.



**Post-Mortem**  
Document, learn, improve.

Communicate Throughout the Entire Process

### COMMON TROUBLESHOOTING COMMANDS

#### KUBERNETES

```
kubectl get pods -A
kubectl describe pod <pod-name> -n <ns>
kubectl logs <pod-name> -n <ns> --tail=200
kubectl get events -A --sort-by=.lastTimestamp
```

#### AZURE CLI

```
az account list
az monitor metrics list --resource <id>
az aks show --name <aks> --resource-group <rg>
az network watcher test-connectivity --...
```

#### LOG ANALYTICS (KQL)

```
requests | where timestamp > ago(1h) |
summarize count() by bin(timestamp, 5m)
exceptions | where timestamp > ago(1h)
```

### RCA WITH 5 WHYS EXAMPLE

- |              |                                                                                               |
|--------------|-----------------------------------------------------------------------------------------------|
| <b>WHY 1</b> | Why did the deployment fail?<br>Because the new pods were not ready.                          |
| <b>WHY 2</b> | Why were the pods not ready?<br>Because the application crashed on startup.                   |
| <b>WHY 3</b> | Why did the application crash?<br>Because it could not connect to the database.               |
| <b>WHY 4</b> | Why could it not connect?<br>Because the database endpoint changed.                           |
| <b>WHY 5</b> | Why did the endpoint change?<br>Because the connection string secret was updated incorrectly. |

**ROOT CAUSE:** Incorrect secret update changed the database endpoint.

### TROUBLESHOOTING CHECKLIST

- ☒ Identify the problem and impacted users.
- ☒ Check recent changes and deployments.
- ☒ Review alerts, dashboards and metrics.
- ☒ Collect logs from all relevant components.
- ☒ Check dependencies and external services.
- ☒ Reproduce issue in lower environment.
- ☒ Identify root cause using 5 Whys.
- ☒ Implement fix or rollback.
- ☒ Validate health and performance.
- ☒ Communicate resolution.
- ☒ Document incident and lessons learned.

**VERIQTA**



YouTube  
/veriqta



GitHub  
/veriqta



LinkedIn  
/company/veriqta



X  
@veriqta



Instagram  
@veriqta\_academy



Telegram  
t.me/veriqta



# MICROSERVICES DELIVERY PIPELINES

MODULAR • SCALABLE • AUTOMATED

## ① SERVICE ISOLATION

- Each service has its own repo
- Independent code, build and pipeline
- Isolated configs, secrets and environments
- Limits blast radius and risk

## ② INDEPENDENT DEPLOYMENTS

- Deploy services independently
- No need to redeploy entire system
- Faster delivery and smaller changes
- Supports continuous delivery

## ③ API VERSIONING

- Version APIs (v1, v2, v3 ...)
- Backward compatibility
- Support multiple versions in parallel
- Deprecate old versions gracefully

## ④ SHARED LIBRARIES

- Common libraries for cross-cutting concerns
- Authentication, logging, utilities, SDKs
- Versioned and published as packages
- Reuse without duplicating code

## ⑤ RELEASE COORDINATION

- Coordinate releases across services
- Contract testing between services
- Feature flags for safe rollouts
- Communicate changes and dependencies

## ⑥ PRODUCTION WORKFLOWS

- CI/CD pipeline per service
- Automated build, test, scan and deploy
- Environments: Dev, Test, Staging, Prod
- Approvals and quality gates

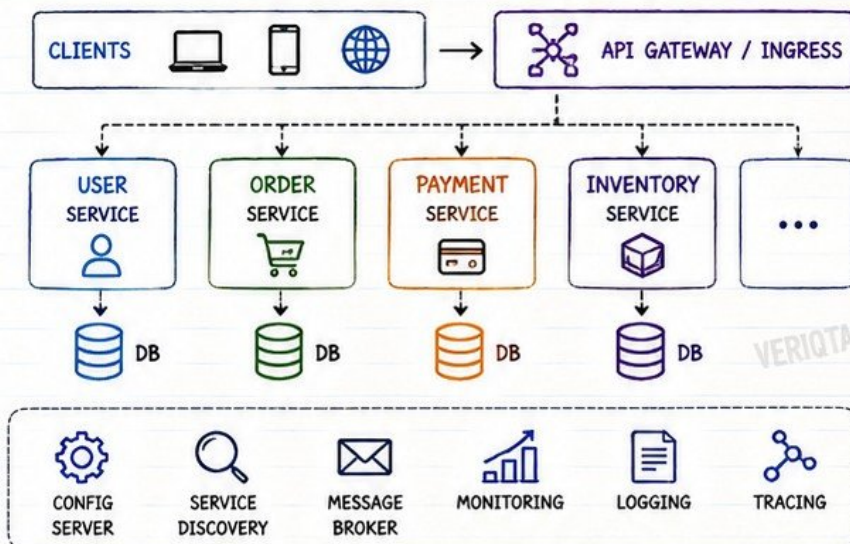
## ⑦ SCALING

- Scale services independently
- Horizontal Pod Autoscaling (HPA)
- Resource limits and requests
- Cost and performance optimized

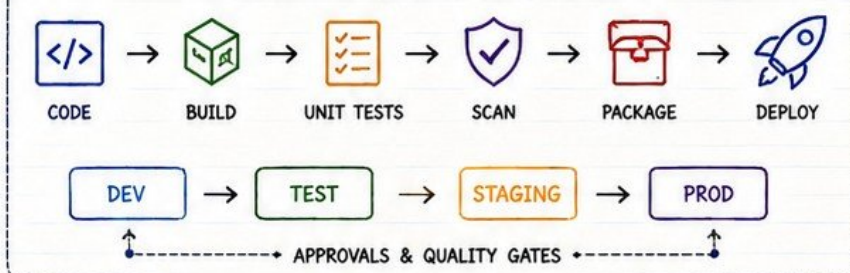
## ⑧ BEST PRACTICES

- Keep services small and focused
- Automate everything
- Observability by default
- Security and compliance built-in

## MICROSERVICES DELIVERY ARCHITECTURE



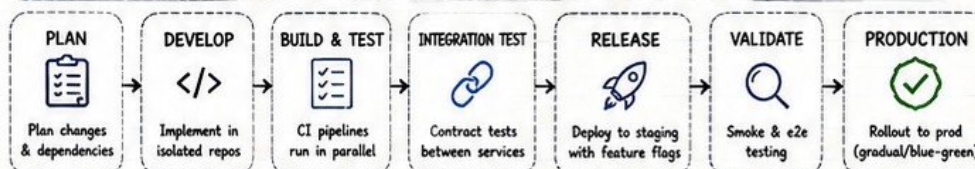
## EXAMPLE: PER-SERVICE CI/CD PIPELINE



## MICROSERVICES PIPELINES BENEFITS



## COORDINATED RELEASE ACROSS MULTIPLE SERVICES

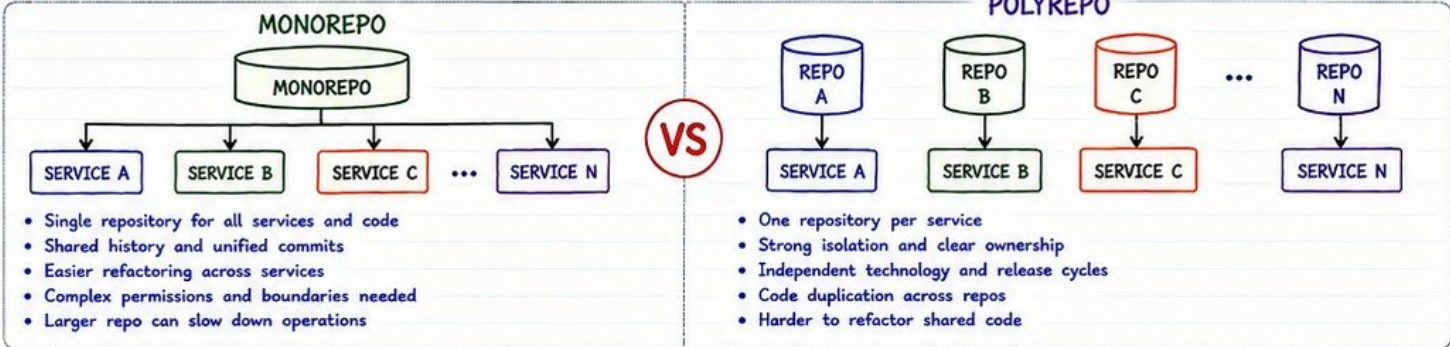


**TIP:** Automate everything, isolate services, monitor continuously and improve relentlessly.

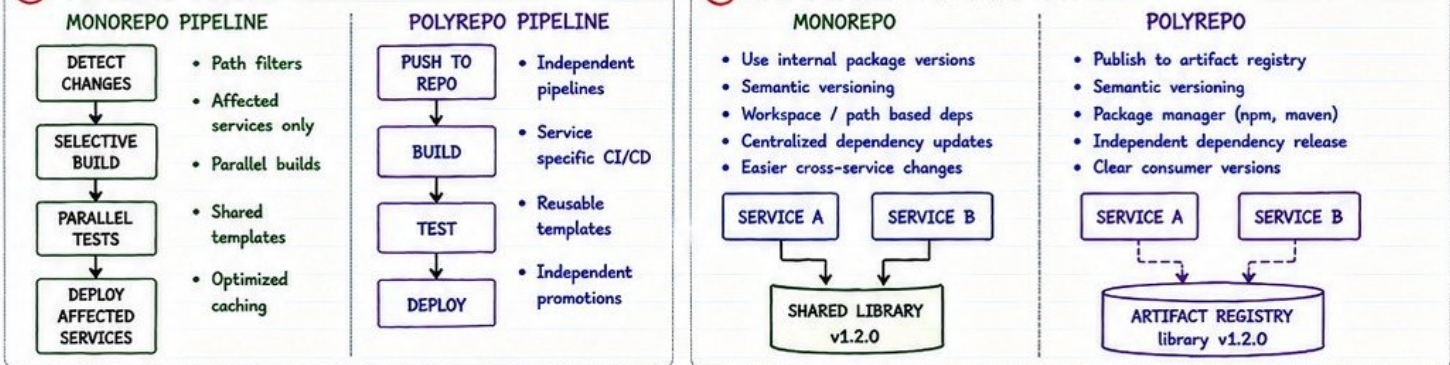
# MONOREPO VS POLYREPO

CHOOSE THE RIGHT MODEL. BUILD AT SCALE. ⚙️

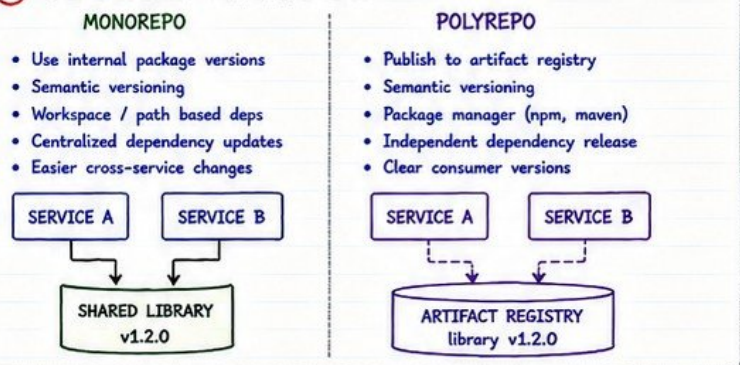
## ① ARCHITECTURE COMPARISON



## ② PIPELINE DESIGN



## ③ DEPENDENCY MANAGEMENT



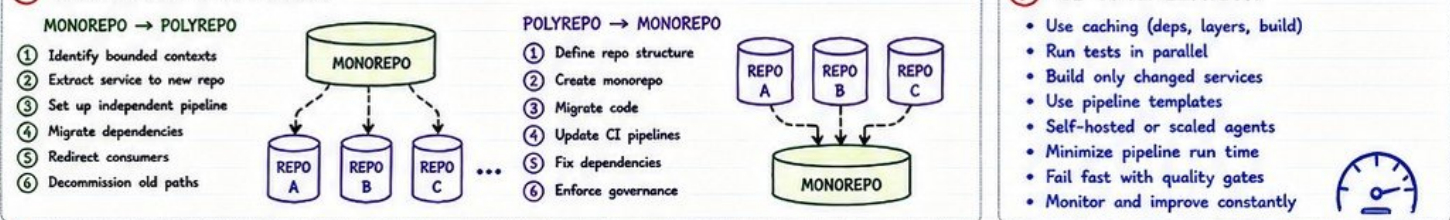
## ④ REPOSITORY GOVERNANCE



## ⑤ ENTERPRISE RECOMMENDATIONS

- ★ Choose monorepo for smaller teams or tight collaboration.
- ★ Choose polyrepo for large orgs with many teams.
- ★ Invest in platform engineering either way.
- ★ Automate everything and enforce standards.
- ★ Optimize CI pipelines from day one.
- ★ Strong governance is critical at scale.

## ⑥ MIGRATION STRATEGIES



## ⑦ CI OPTIMIZATION

- Use caching (deps, layers, build)
  - Run tests in parallel
  - Build only changed services
  - Use pipeline templates
  - Self-hosted or scaled agents
  - Minimize pipeline run time
  - Fail fast with quality gates
  - Monitor and improve constantly
- 

## ⑧ BEST PRACTICES

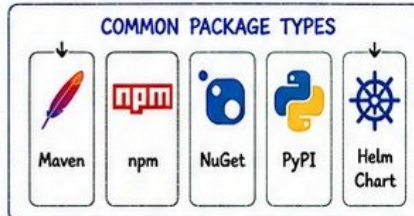


# ENTERPRISE ARTIFACT MANAGEMENT

STORE • MANAGE • SECURE • PROMOTE

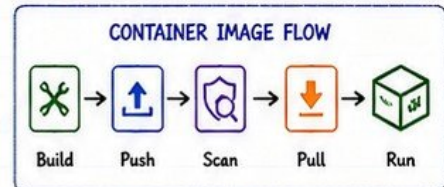
## ① UNIVERSAL PACKAGES

- Support many package types
- One repository, many formats
- Share artifacts across teams
- Standardize and automate
- Secure and scalable



## ② CONTAINER IMAGES

- Store container images
- Scan for vulnerabilities
- Immutable and versioned
- Promote across environments
- High availability and HA



## ③ PACKAGE LIFECYCLE



## ④ ARTIFACT RETENTION

- Define retention policies
- Balance cost and compliance
- Automate cleanup
- Protect important artifacts

### EXAMPLE RETENTION POLICY

| ARTIFACT TYPE      | KEEP    |
|--------------------|---------|
| Release Artifacts  | 90 days |
| Snapshot Builds    | 30 days |
| CI Build Artifacts | 7 days  |
| Logs / Reports     | 14 days |
| Container Images   | 60 days |
| Helm Charts        | 90 days |

### STRATEGIES



## ⑤ VERSIONING

- Use semantic versioning
- Immutable versions
- Metadata and checksums
- Easy rollback and traceability

### SEMANTIC VERSIONING (SEMVER)

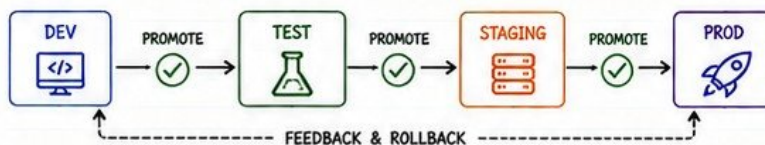


### RULES

- MAJOR: Incompatible changes
- MINOR: Backward compatible features
- PATCH: Backward compatible bug fixes
- BUILD: Build metadata (optional)

## ⑥ PROMOTION FLOW

- Promote artifacts across environments
- Enforce quality gates
- Traceable and repeatable
- No rebuilds in higher environments



## ⑦ REPOSITORY GOVERNANCE

- Define access and roles
- Enforce naming standards
- Immutable releases
- Audit and compliance
- Secure credentials and APIs



## ⑧ PRODUCTION PRACTICES



## ⑨ KEY BENEFITS

- ★ Faster and reliable deployments
- ★ Consistency across environments
- ★ Better security and compliance
- ★ Reduced storage cost
- ★ Traceability and audit readiness
- ★ Improved team productivity



**TIP:**

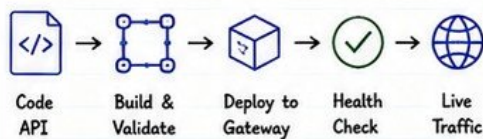
Build once, store safely, promote with confidence.  
A strong artifact strategy is the backbone of enterprise DevOps.



## 1 API DEPLOYMENT

- Automate API build and deployment
- Consistent and repeatable releases
- Blue/Green and Canary deployments
- Rollback on failure
- Zero-downtime strategy

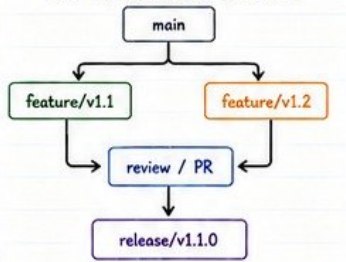
### API DEPLOYMENT FLOW



## 2 VERSION CONTROL

- Store API specs in Git
- Track changes and history
- Branching for features
- Pull requests for review
- Tag releases for versions

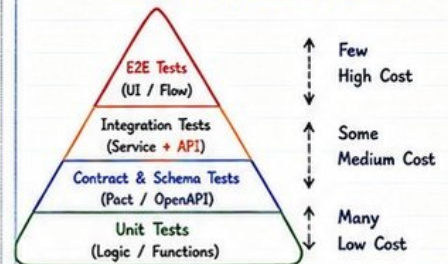
### GIT WORKFLOW FOR APIS



## 3 API TESTING

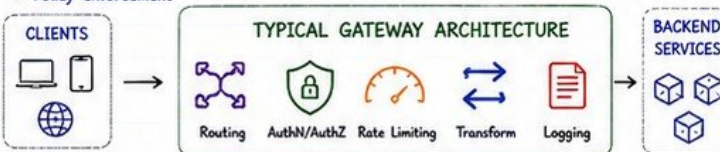
- Automated functional tests
- Schema validation
- Contract testing
- Performance & load testing
- Security testing

### TESTING PYRAMID FOR APIS



## 4 GATEWAY INTEGRATION

- Centralized API gateway
- Routing, transformation, throttling
- Authentication & authorization
- Rate limiting & quotas
- Policy enforcement



## 5 SECURITY

- OAuth2 / OpenID Connect
- API Keys & JWT Tokens
- mTLS for service-to-service
- Input validation & sanitization
- Threat protection (WAF, DDoS)

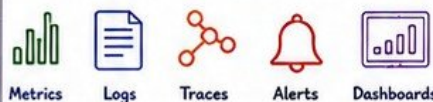
### SECURITY LAYERS



## 6 MONITORING

- Real-time API analytics
- Performance monitoring
- Error tracking & alerts
- SLA / SLO monitoring
- Business metrics & usage

### API OBSERVABILITY STACK



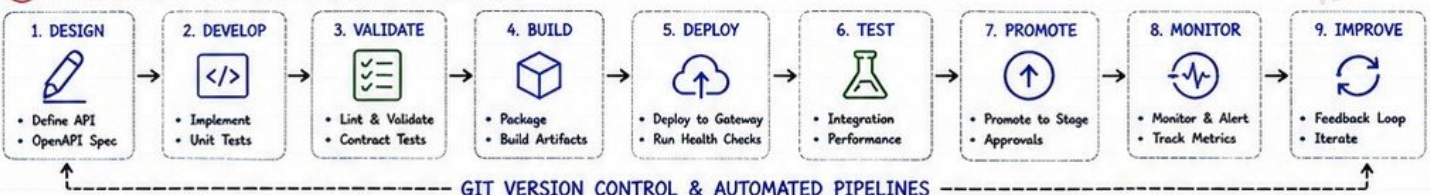
## 7 AUTOMATION

- Infrastructure as Code
- API spec validation in CI
- Automated deployments
- Policy as Code
- Self-service API pipelines

### AUTOMATION TOOLS



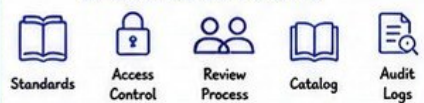
## 8 ENTERPRISE API MANAGEMENT WORKFLOW



## 9 REPOSITORY GOVERNANCE

- Standard naming conventions
- Access control & roles
- Review & approval process
- API catalog & documentation
- Compliance & audit trail

### GOVERNANCE COMPONENTS



## 10 PRODUCTION BEST PRACTICES

- ★ Design APIs for consumers
- ★ Version APIs thoughtfully
- ★ Automate everything
- ★ Monitor, alert and act fast
- ★ Secure by default
- ★ Document APIs clearly
- ★ Enforce quality gates
- ★ Manage lifecycle
- ★ Optimize performance
- ★ Continuously improve

**TIP:** Automate API management end-to-end for reliability, security and developer velocity. ✓

Automate, Version, Validate and Deliver Databases to Production

DATA + DEVOPS 

## 1 DATABASE PIPELINES



- Treat databases as code
- Automate build, test, deploy
- Integrate with CI/CD pipelines
- Ensure consistency and repeatability
- Reduce manual errors and risk

## 2 SCHEMA MIGRATIONS



- Apply incremental schema changes
- Track changes over time
- Up and Down migrations
- Prevent drift across environments
- Safe, repeatable deployments

## 3 DACPAC (SQL SERVER)



- Schema as a DACPAC file
- Published from SSDT or DacFx
- Deploy to any environment
- Supports drift detection
- Ideal for SQL Server databases

## 4 ROLLBACK STRATEGY



- Always have a rollback plan
- Use transaction scripts
- Backup before deployment
- Down migrations for schema
- Fast recovery with automation

### TYPICAL DATABASE DEVOPS PIPELINE



1. Commit  
(DB Changes)



2. Build  
(DACPAC / Scripts)



3. Validate  
(Tests & Checks)



4. Deploy  
(Target Environment)



5. Verify  
(Post-Deployment)

### INTEGRATED WITH AZURE DEVOPS



Azure Repos



Azure Pipelines



Artifacts /  
Azure Artifacts



Environments



Approvals &  
Checks



Notifications

### SCHEMA MIGRATION FLOW



Dev DB



Migration  
Script



Test DB



Migration  
Script



Prod DB

### FEATURES

- ✓ Version controlled changes
- ✓ Automated execution
- ✓ Up and Down migrations
- ✓ Drift detection
- ✓ Environment consistency
- ✓ Rollback support

### DATABASE RELEASE STRATEGY



1. DEV → Developer builds and tests locally



2. TEST → Automated tests & validation



3. STAGING → Pre-production validation



4. PRODUCTION → Approved & automated deployment

## VALIDATION



- Schema validation (syntax & rules)
- Data integrity checks
- Pre-deployment tests
- Post-deployment verification
- Automated quality gates

## 6 VERSIONING



- Version schema changes
- Use branching strategy
- Tag releases
- Track applied versions
- Maintain change history

## 7 PRODUCTION DATABASE DEPLOYMENTS



- Use least privilege accounts
- Apply changes during low traffic windows
- Monitor performance and errors
- Backup before deployment
- Ensure zero-downtime where possible

### EXAMPLE: DEPLOY DACPAC USING AZURE DEVOPS

```
- task: SqlAzureDacpacDeployment@1
  displayName: 'Deploy DACPAC'
  inputs:
    azureSubscription: '$(azureServiceConnection)'
    AuthenticationType: 'server'
    ServerName: '$(dbServer)'
    DatabaseName: '$(dbName)'
    DacpacFile: '$(Pipeline.Workspace)/drop/Database.dacpac'
    DeploymentAction: 'Publish'
    IpDetectionMethod: 'AutoDetect'
```

### BEST PRACTICES

- ✓ Automate everything
- ✓ Version your database
- ✓ Validate before deploy
- ✓ Always backup
- ✓ Use approvals for prod
- ✓ Monitor and alert
- ✓ Keep rollbacks ready
- ✓ Document changes



# CLOUD AUTOMATION

Automate. Orchestrate. Operate. Optimize.

AUTOMATE EVERYTHING



Azure CLI

## ① AZURE CLI

- Command-line automation for Azure
- Manage resources, deployments
- Scripting & CI/CD integration
- Cross-platform and powerful



PowerShell

## ② POWERSHELL

- Advanced automation with PowerShell
- Azure PowerShell modules
- Object-based & extensible
- Great for enterprise automation



{ REST API }

## ③ REST API

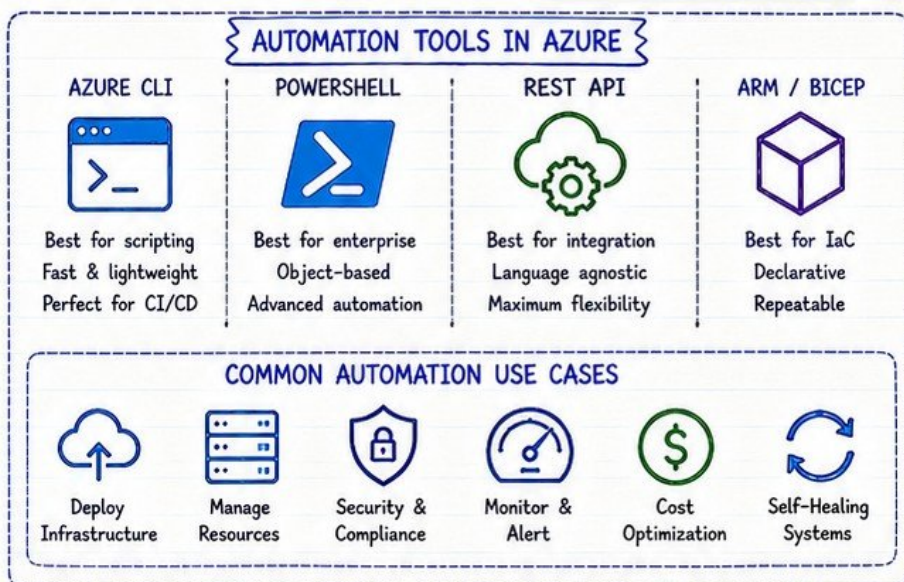
- Integrate using Azure REST APIs
- Full control and flexibility
- Use with any language or tool
- Ideal for custom integrations



Scripts

## ④ AUTOMATION SCRIPTS

- Bash, PowerShell, Python, or Go
- Reusable and version-controlled
- Parameterized & modular scripts
- Centralized script repositories



## ⑤ SCHEDULING

- Azure Automation Runbooks
- Logic Apps + Schedule Trigger
- Time-based task execution
- CRON-like scheduling



## ⑦ SELF-HEALING

- Detect issues automatically
- Trigger corrective actions
- Auto restart, scale, replace
- Reduce downtime, improve resilience

### AZURE CLI EXAMPLE

```
# Create a Resource Group
az group create \
  --name myRG \
  --location eastus

# Start a VM
az vm start \
  --resource-group myRG \
  --name myVM

# List all Storage Accounts
az storage account list \
  --query "[].name" -o table
```

### POWERSHELL EXAMPLE

```
# Connect to Azure
Connect-AzAccount

# Create Resource Group
New-AzResourceGroup \
  -Name 'myRG' \
  -Location 'EastUS'

# Get all VMs
Get-AzVM -ResourceGroupName myRG \
  | Select-Object Name, PowerState

# Stop a VM
Stop-AzVM -Name myVM -ResourceGroupName myRG
```

### REST API EXAMPLE

```
# Get all Resource Groups
GET https://management.azure.com/
subscriptions/{subId}/resourcegroups
?api-version=2021-04-01

# Create Resource Group
PUT https://management.azure.com/
subscriptions/{subId}/resourcegroups/
myRG?api-version=2021-04-01

Body:
{ "location": "EastUS", "tags": { } }
```

### BASH SCRIPT EXAMPLE

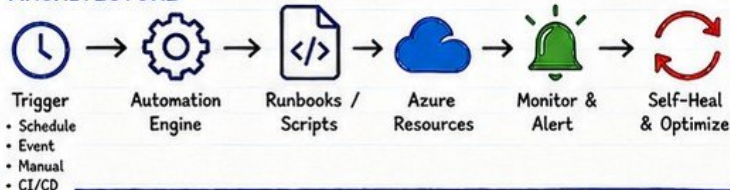
```
#!/bin/bash
RG="myRG"
LOCATION="eastus"

az group create \
  --name $RG \
  --location $LOCATION

az vm start \
  --resource-group $RG \
  --name myVM

echo "Automation complete!"
```

### AUTOMATION ARCHITECTURE



### PRODUCTION AUTOMATION BEST PRACTICES

- ✓ Store scripts in source control (Git)
- ✓ Use Managed Identities, not stored credentials
- ✓ Implement role-based access control (RBAC)
- ✓ Log all actions and monitor execution
- ✓ Test automation in lower environments first
- ✓ Make automation idempotent and repeatable
- ✓ Implement alerts and self-healing workflows



**TIP:** Automate everything that is repeatable. Consistency, speed and reliability come from automation.

# ENTERPRISE NETWORKING

Design. Secure. Connect. Validate. Deliver.

SECURE BY DESIGN



## 1 VIRTUAL NETWORKS

- Build isolated and scalable networks
- Address spaces & subnet design
- Hub-and-spoke architecture
- Route tables and UDRs
- Service endpoints integration



## 2 PRIVATE ENDPOINTS

- Private access to PaaS services
- Eliminate public exposure
- DNS integration required
- Granular and secure connectivity
- Works over Private Link



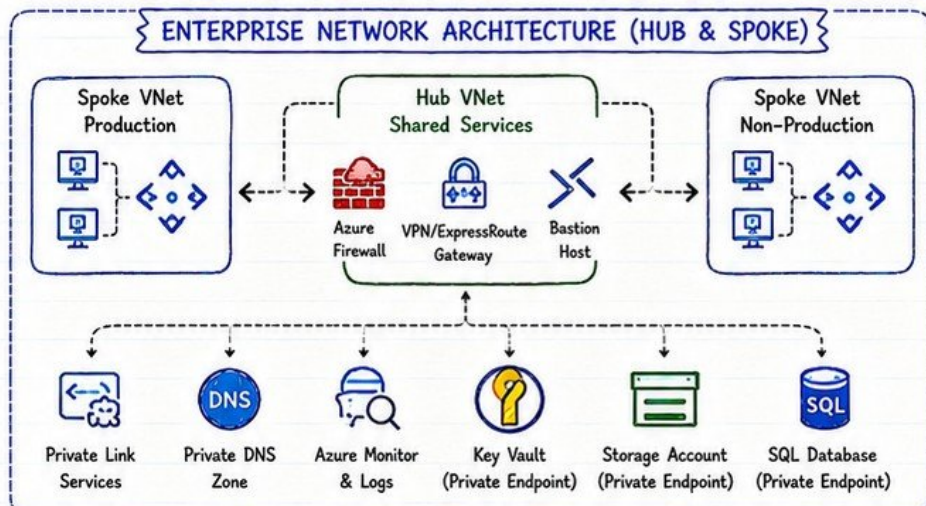
## 3 DNS (AZURE DNS)

- Private DNS zones
- DNS forwarding & resolution
- Custom domain integration
- Split-horizon DNS design
- Name resolution validation



## 4 FIREWALLS

- Azure Firewall for central filtering
- Application & network rules
- Threat intelligence & logging
- DNAT, SNAT & forced tunneling
- High availability & scalability



## 5 NSGs (NETWORK SECURITY GROUPS)



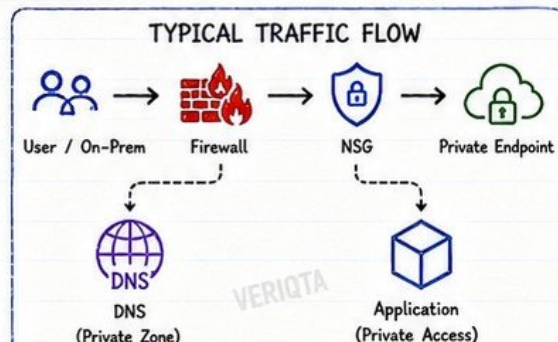
- Control traffic at subnet & NIC level
- Inbound and outbound rules
- Default deny all
- Priority-based rule evaluation
- Flow logs for visibility

## 6 CONNECTIVITY VALIDATION



- Validate DNS resolution
- Test reachability & ports
- Check routes & next hops
- Verify NSG & Firewall rules
- End-to-end connectivity tests

## NETWORK SECURITY LAYERS



## BEST PRACTICES FOR PRODUCTION NETWORKING

- ✓ Use Hub and Spoke for scalability and isolation
- ✓ Centralize logging and monitoring
- ✓ Document network architecture and IP planning
- ✓ Use Private Endpoints for all PaaS service access
- ✓ Use Azure DNS Private Zones
- ✓ Test failover scenarios for gateways and firewalls
- ✓ Enforce least privilege with NSGs and Firewall rules
- ✓ Regularly validate connectivity and DNS resolution
- ✓ Continuously review and harden network security

## SECURE NETWORKING PRINCIPLES

- 🔒 Minimize Exposure
- 🛡️ Enforce Least Privilege
- 👁️ Monitor & Detect
- 🔄 Automate & Validate
- 📝 Document & Improve



**TIP:**

Every connection should be intentional, encrypted, and monitored.  
Build networks that are resilient, observable, and secure by design.



Secure Access. Verify Everything. Empower Workloads.

IDENTITY IS THE NEW PERIMETER



## 1 MICROSOFT ENTRA ID

- Cloud identity and access management
- Centralized user and group management
- SSO to Microsoft & 3rd party apps
- Conditional Access & risk-based access
- Lifecycle management & governance



## 2 MANAGED IDENTITY

- Identity for Azure resources
- No secrets in code or configuration
- System-assigned & user-assigned
- Auto rotation and secure by default
- Used with Azure services seamlessly



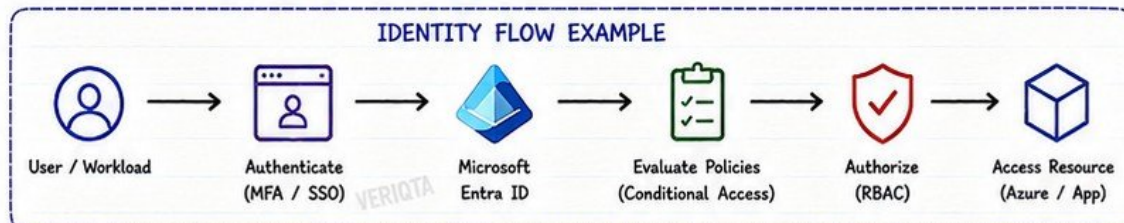
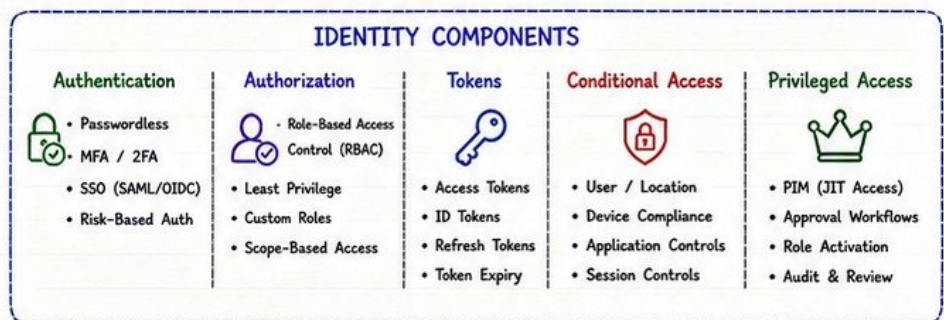
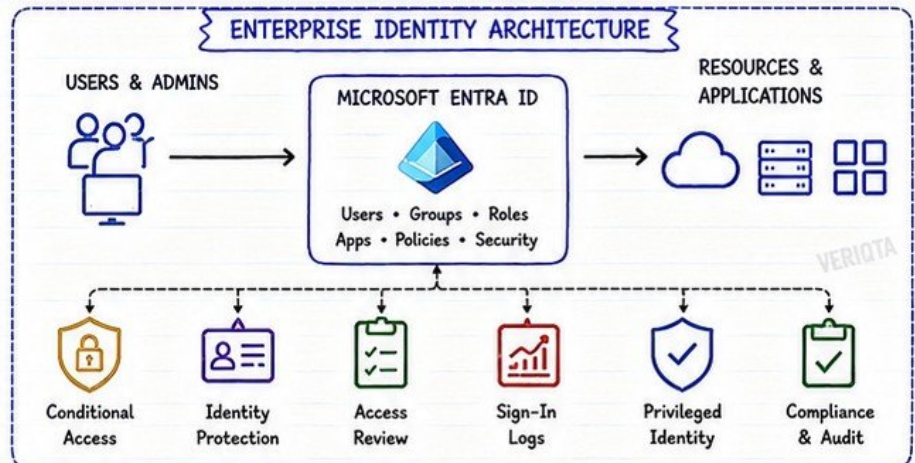
## 3 SERVICE PRINCIPALS

- Application identity for automation
- Access Azure via APIs and CLI/SDK
- Assign roles to resources & scopes
- Used for DevOps, CI/CD, and apps
- Secret or certificate authentication



## 4 WORKLOAD IDENTITY

- Federated identity for workloads
- No secrets or certificates required
- OIDC-based authentication
- Used with Kubernetes and AKS
- Improve security for cloud-native apps



### 5 AUTHENTICATION

- Azure AD SSO
- Multi-Factor Authentication
- Passwordless & FIDO2
- Smart Lockout & Protection
- Session & Token Management

### 6 AUTHORIZATION

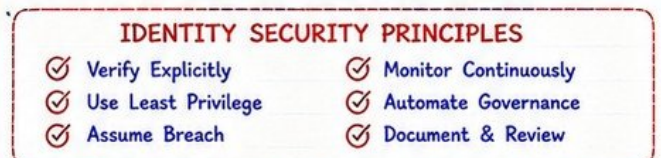
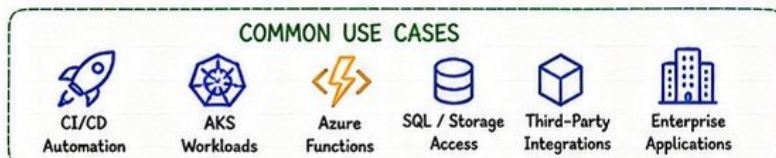
- RBAC for Azure Resources
- Azure AD Roles
- Scoped Access Control
- Attribute-Based Access
- Just-In-Time Access

### 7 ENTERPRISE GOVERNANCE

- Identity Lifecycle Management
- Access Reviews & Certifications
- Role & Policy Governance
- Logging, Monitoring & Audit
- Compliance (ISO, SOC, NIST)

### 8 BEST PRACTICES

- Enforce Least Privilege
- Use Managed Identities
- Enable Conditional Access
- Monitor & Alert on Sign-Ins
- Review & Remove Unused Access



**TIP:** Strong identity is the foundation of a secure and modern cloud enterprise.

Build Platforms. Empower Developers. Drive Product Velocity.

PLATFORM = PRODUCT 

**1 INTERNAL DEVELOPER PLATFORMS**

- Abstract complexity and add value
- Provide self-service for developers
- Improve velocity, reliability and security
- Treat platforms as products
- Focus on developer experience

  
Internal Developer Platform

**2 GOLDEN PATHS**

- Pre-approved, secure and scalable
- Paved paths for common use cases
- Reduce cognitive load and risk
- Built with guardrails and automation
- Accelerate delivery with confidence

  
Golden Path

**3 SELF-SERVICE INFRASTRUCTURE**

- On-demand environments and resources
- No manual tickets or handoffs
- Catalog-driven infrastructure
- Role-based access and approvals
- Built-in policies and cost controls

  
Self-Service Infrastructure

**4 STANDARDIZATION**

- Standard tools, patterns and templates
- Consistent environments and configurations
- Reusable components and modules
- Reduce sprawl and technical debt
- Improve quality and governance

  
Standardization

**5 AUTOMATION**

- Automate provisioning, config and operations
- GitOps and policy as code
- Event-driven and AI-assisted automation
- Reduce toil and manual work
- Fast, repeatable and reliable

  
Automation

**6 PLATFORM APIS**

- Well-designed and documented APIs
- Secure, versioned and scalable
- Enable integration and extensibility
- Expose platform capabilities
- Developer-friendly by default

  
Platform APIs

**7 DEVELOPER EXPERIENCE**

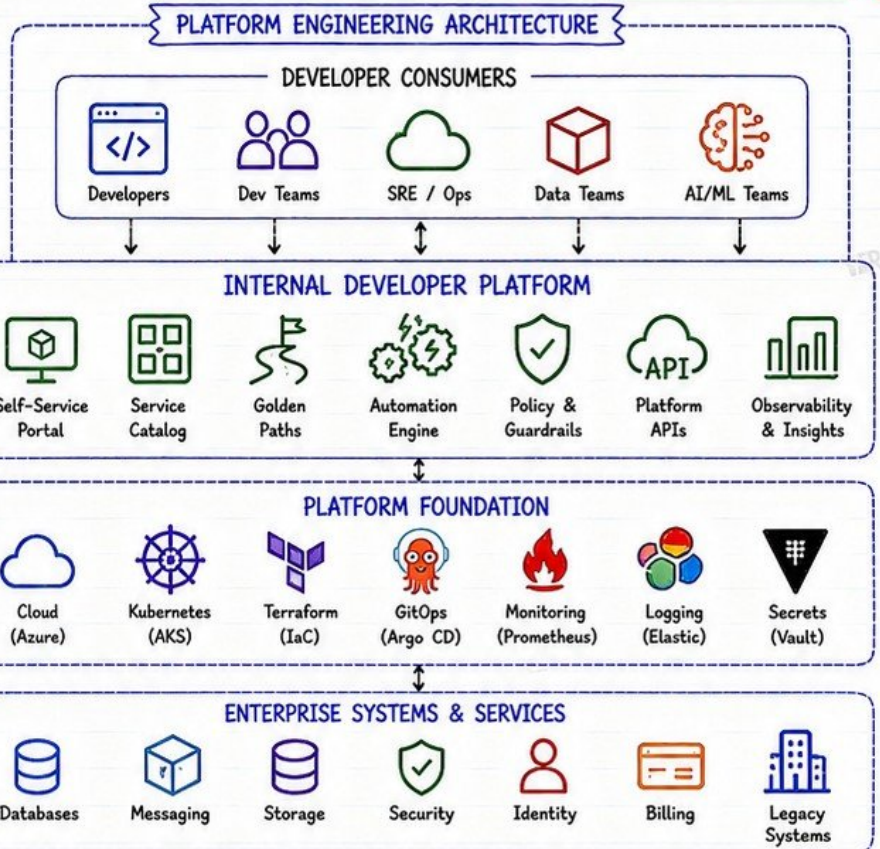
- Intuitive and easy to use
- Fast onboarding and feedback
- Clear docs and examples
- Self-service with visibility
- Delight developers every day

  
Developer Experience

**8 ENTERPRISE ARCHITECTURE**

- Align with business capabilities
- Scalable, resilient and secure
- Governance, compliance and risk
- Integration across ecosystems
- Evolve with the organization

  
Enterprise Architecture



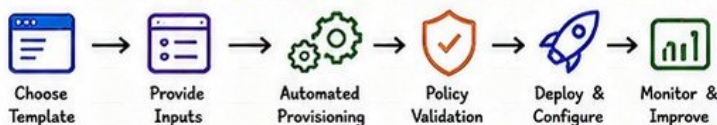
## PLATFORM CAPABILITIES



## PLATFORM PRINCIPLES

- ✓ Platform as a Product
- ✓ You Build It, You Run It
- ✓ Self-Service, Not DIY
- ✓ Automation First
- ✓ Secure by Default
- ✓ Measure & Improve

## TYPICAL GOLDEN PATH EXAMPLE



## PLATFORM OUTCOMES



 **TIP:** A great platform removes friction, not freedom.



Design Enterprise-Scale. Govern Effectively. Deliver Consistently.

ARCHITECT FOR SCALE



Subscriptions

## 1 MULTI-SUBSCRIPTION DESIGN

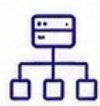
- Separate subscriptions by environment and workload
- Isolate billing, access and policies
- Improve security and governance
- Enable independent team ownership



Landing Zone

## 2 LANDING ZONES

- Pre-configured, secure environments
- Standardized subscriptions, networks, identities and policies
- Accelerate onboarding of workloads
- Built on Microsoft Cloud Adoption Framework (CAF)



Management Groups

## 3 MANAGEMENT GROUPS

- Organize subscriptions at scale
- Apply policies & access at hierarchy
- Inherit governance and compliance
- Enable centralized management



Shared Services

## 4 SHARED SERVICES

- Centralized services for the enterprise
- Identity, networking, security, monitoring, backup
- Reduce duplication and cost
- Improve security and consistency



## 6 GOVERNANCE

- Policy-driven governance
- Role-based access control
- Compliance and audit
- Cost management
- Tagging and standards



## 7 ENTERPRISE STANDARDS

- Naming conventions
- Tagging strategy
- Resource location rules
- Security baselines
- Configuration standards



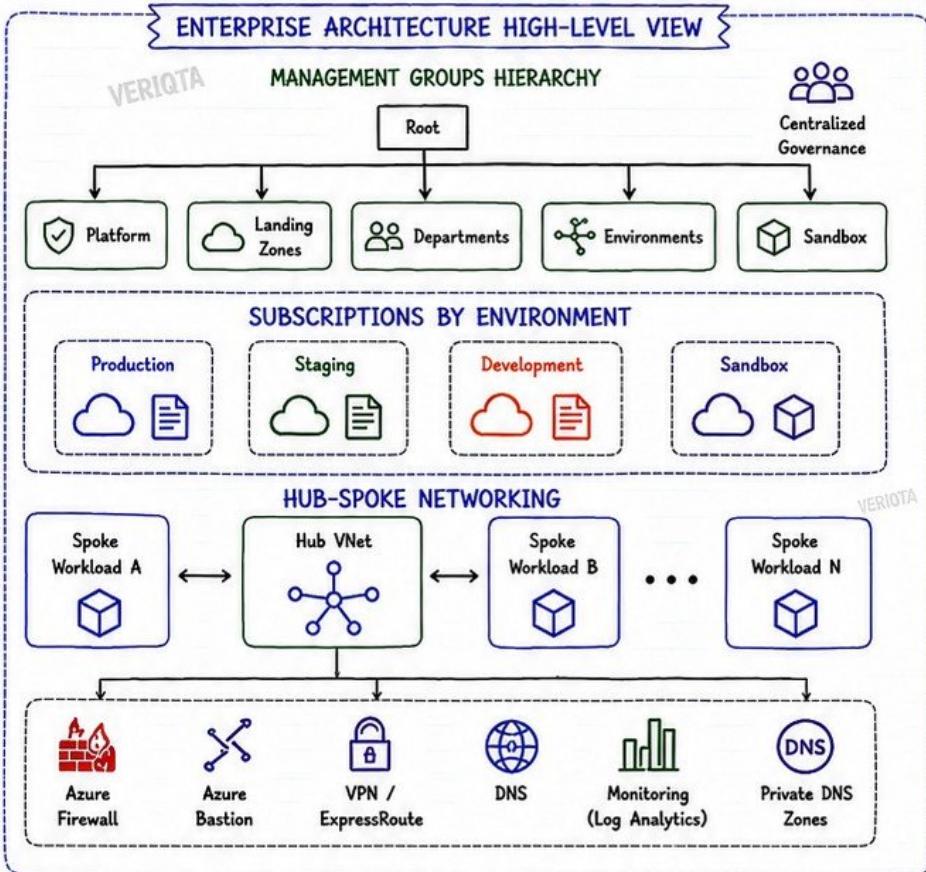
## 8 COMPLIANCE & RISK

- Regulatory compliance
- Continuous assessment
- Risk management
- Audit and reporting
- Data protection

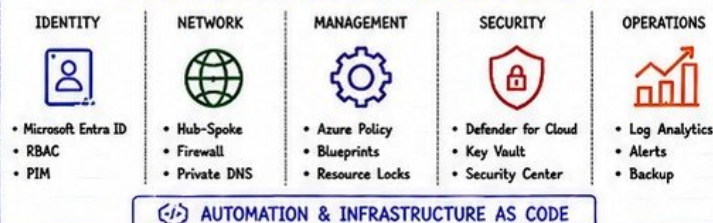


## 9 BEST PRACTICES

- Start with a Landing Zone
- Automate everything
- Follow least privilege
- Monitor and optimize
- Continuously improve



## TYPICAL ENTERPRISE LANDING ZONE DESIGN

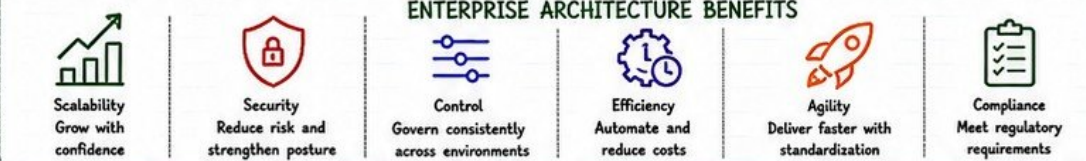


## ENTERPRISE ARCHITECTURE PRINCIPLES

- ✓ Design for Scale and Resilience
- ✓ Secure by Default
- ✓ Automate and Standardize
- ✓ Centralize Where It Makes Sense
- ✓ Delegate Ownership with Guardrails
- ✓ Monitor, Measure and Improve



## ENTERPRISE ARCHITECTURE BENEFITS



## TIP:

Strong architecture today ensures secure, scalable and sustainable growth tomorrow.



## 1 MULTI-REGION DEPLOYMENTS

- Deploy workloads in multiple Azure regions
- Global traffic management with Azure Front Door or Traffic Manager
- Active-Active or Active-Passive architectures
- Data replication across regions
- Reduce latency and improve user experience



## 2 DISASTER RECOVERY

- Prepare for regional outages and major failures
- Define RPO (Recovery Point Objective)
- Define RTO (Recovery Time Objective)
- Maintain DR environment in paired region
- Document DR runbooks and procedures



## 3 FAILOVER

- Automatic or manual failover
- Health-based failover using monitoring
- DNS or traffic routing failover
- Database failover (Geo-Replication)
- Test failover regularly



## 4 BACKUP

- Automated backups for all critical data
- Use Azure Backup, Azure SQL Backup, Storage Redundancy
- Geo-redundant storage (RA-GRS)
- Backup retention policies
- Immutable backups for ransomware protection



## 5 RECOVERY TESTING

- Perform DR drills and simulations
- Validate backups and restore procedures
- Test failover end-to-end
- Measure RTO and RPO
- Improve based on test results



## 6 RESILIENCE

- Remove single points of failure
- Use Availability Zones and Redundant Services
- Auto-scaling and self-healing systems
- Monitor health and capacity continuously
- Design for graceful degradation



## 7 BUSINESS CONTINUITY

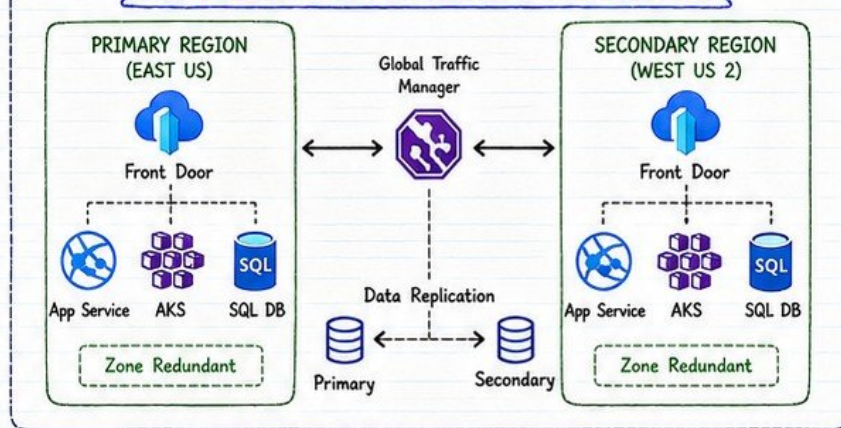
- Ensure critical business functions continue
- Alternative access and communication plans
- Regularly review and update BCP
- Align IT strategy with business goals
- Minimize downtime and financial impact



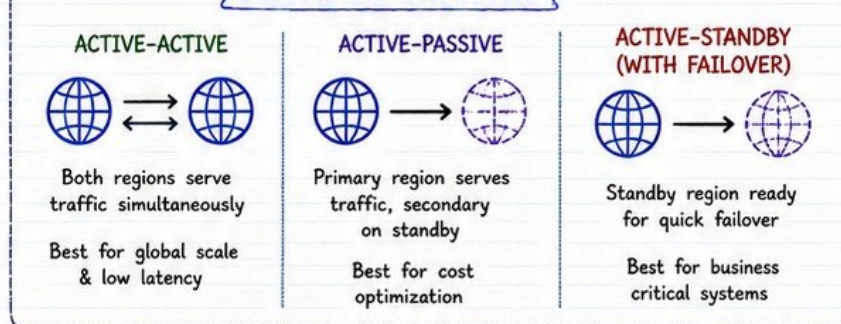
## 8 ENTERPRISE DESIGN PRINCIPLES

- Multi-region by design
- Automate everything
- Security and compliance built-in
- Observability and alerting everywhere
- Test, document, improve continuously

### MULTI-REGION HIGH AVAILABILITY ARCHITECTURE



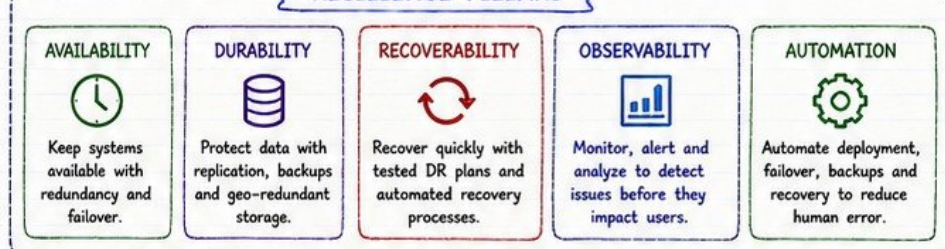
### DEPLOYMENT STRATEGIES



### DISASTER RECOVERY WORKFLOW



### RESILIENCE PILLARS



**TIP:** Test your failover, backups, and recovery plans regularly. What you don't test in calm, you will fail in chaos.



OPTIMIZE EVERYTHING. DELIVER FASTER. SCALE SMARTER.

## 1 PIPELINE OPTIMIZATION

- Parallelize stages and jobs
- Use caching and artifacts smartly
- Minimize stage dependencies
- Reuse templates and components
- Fail fast with early validations
- Optimize triggers and schedules

## 2 BUILD OPTIMIZATION

- Enable build caching (Maven, npm, pip, Docker)
- Incremental builds and change detection
- Use lightweight base images
- Optimize dependency management
- Use multi-stage builds
- Avoid unnecessary rebuilds

## 3 DEPLOYMENT OPTIMIZATION

- Use rolling / blue-green / canary strategies
- Reduce deployment frequency time
- Pre-deploy validation and smoke tests
- Optimize Helm charts and manifests
- Use automation for approvals
- Minimize downtime and blast radius

## 4 AGENT SCALING

- Use self-hosted agents with auto-scaling
- Scale agents based on queue demand
- Use containerized/Kubernetes agents
- Separate agents by workload type
- Monitor agent utilization
- Keep agents updated and healthy

## 5 COST OPTIMIZATION

- Right-size agents and compute
- Use spot instances for non-critical jobs
- Clean up unused resources
- Optimize storage and retention policies
- Monitor cost continuously
- Apply tagging and cost governance

## 6 RESOURCE OPTIMIZATION

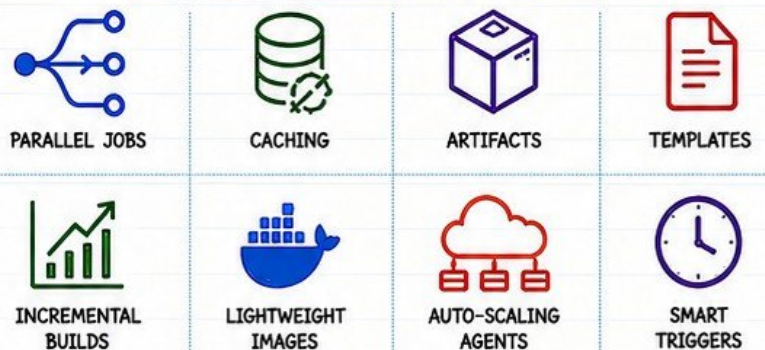
- Optimize CPU and memory requests/limits
- Use efficient container images
- Optimize Kubernetes scheduling
- Reduce idle resources
- Tune databases and caches
- Balance performance and cost

### PERFORMANCE-DRIVEN PIPELINE FLOW

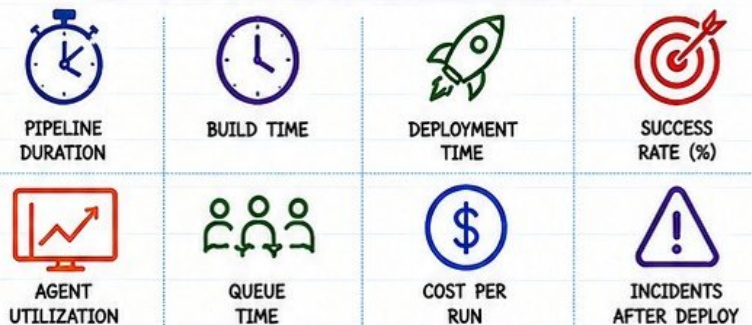


- ✓ FASTER
- ✓ RELIABLE
- ✓ SCALABLE
- ✓ COST-EFFECTIVE

### KEY OPTIMIZATION TECHNIQUES



### PERFORMANCE METRICS TO TRACK



### BEST PRACTICES

- ✓ Automate everything
- ✓ Measure everything
- ✓ Eliminate bottlenecks
- ✓ Optimize for feedback speed
- ✓ Use infrastructure as code
- ✓ Standardize pipelines and templates
- ✓ Stay updated with best practices
- ✓ Test performance regularly
- ✓ Design for scalability
- ✓ Continuously improve



**TIP:** Performance is not a one-time task.  
It is a continuous journey of measurement, optimization, and improvement.



# COMPLIANCE AUTOMATION

AUTOMATE COMPLIANCE. REDUCE RISK. PROVE TRUST.

## 1 ISO 27001

- Information Security Management System
- Risk-based approach
- Annex A controls
- Continuous improvement (PDCA)
- Statement of Applicability (SoA)

## 2 SOC 2

- Trust Services Criteria (TSC)
- Security, Availability, Confidentiality, Processing Integrity, Privacy
- Common Criteria framework
- Continuous monitoring
- Auditor attestation

## 3 PCI DSS

- Cardholder data protection
- 12 core requirements
- Network security controls
- Vulnerability management
- Regular testing and monitoring

## 4 GDPR

- Data protection and privacy
- Lawful basis for processing
- Data subject rights
- Data minimization
- Breach notification (72 hours)

## 5 AUDIT EVIDENCE

- Collect evidence automatically
- Centralize and organize
- Immutable and time-stamped
- Link evidence to controls
- Ready for auditor review

## 6 CONTINUOUS COMPLIANCE

- Monitor controls 24/7
- Detect drift and misconfigurations
- Auto-remediate where possible
- Continuous control validation
- Reduce audit fatigue

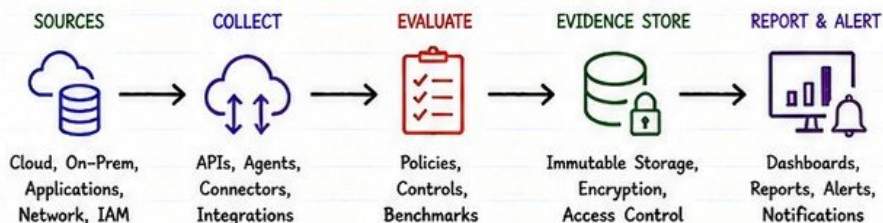
## 7 REPORTING

- Real-time compliance dashboards
- Executive summaries
- Control coverage reports
- Exception and risk reports
- Scheduled and ad-hoc reports

## 8 ENTERPRISE GOVERNANCE

- Policies, standards, procedures
- Roles and responsibilities
- Risk and compliance oversight
- Governance committees
- Metrics and KPIs

### COMPLIANCE AUTOMATION ARCHITECTURE



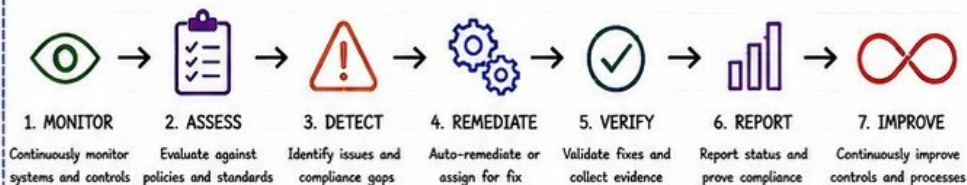
### KEY CAPABILITIES



### STANDARDS AT A GLANCE

|           | ISO 27001                           | SOC 2                                               | PCI DSS                                      | GDPR                                                  |
|-----------|-------------------------------------|-----------------------------------------------------|----------------------------------------------|-------------------------------------------------------|
| FOCUS     | Information Security Management     | Trust and Service Integrity                         | Payment Card Data Security                   | Data Protection and Privacy                           |
| KEY AREAS | Policies, Controls, Risk Management | TSC: Security, Avail., Confidentiality, PI, Privacy | Network, Access, Monitoring, Testing, Policy | Lawfulness, Rights, Security, Transfers, Breach Mgmt. |
| AUDITS    | Internal + External Certification   | Independent Auditor Attestation                     | Qualified Security Assessor (QSA)            | Regulatory Authorities                                |
| FREQUENCY | Annually                            | Annually                                            | Annually + ASV Scans Quarterly               | Continuous                                            |
| SCOPE     | Organization-wide                   | Systems and Services                                | Cardholder Data Environment                  | Personal Data Processing                              |

### CONTINUOUS COMPLIANCE LIFECYCLE



**TIP:**

Automate evidence collection, standardize controls, and monitor continuously. Compliance is not a project — it is an operational capability.

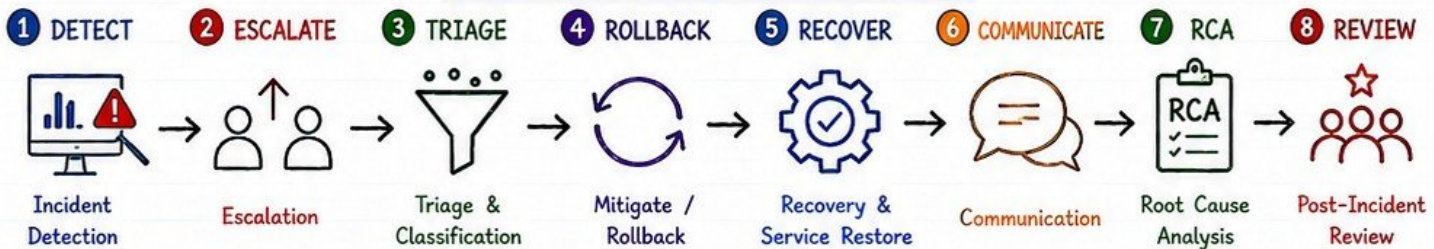


# PRODUCTION INCIDENT RESPONSE

ENTERPRISE  
GOVERNANCE

FAST DETECTION. RAPID RESPONSE. MINIMAL IMPACT.  
RECOVER STRONGER.

## INCIDENT RESPONSE LIFECYCLE



### KEY PRINCIPLES



- People first, then systems
- Act fast, communicate early
- Data-driven decisions
- Minimize customer impact
- Document everything
- Continuous improvement

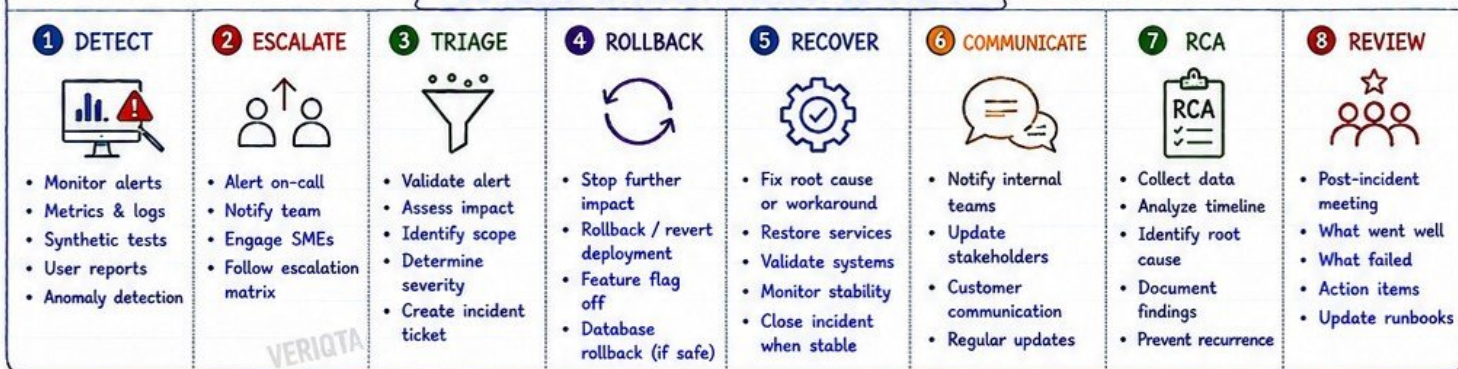
### INCIDENT SEVERITY LEVELS

|           |                 |                             |
|-----------|-----------------|-----------------------------|
| <b>P1</b> | <b>CRITICAL</b> | Major outage, severe impact |
| <b>P2</b> | <b>HIGH</b>     | Significant degradation     |
| <b>P3</b> | <b>MEDIUM</b>   | Minor impact                |
| <b>P4</b> | <b>LOW</b>      | Minimal or no user impact   |

### COMMUNICATION CHANNELS

-  Status Page
-  Slack / Teams
-  Email Notifications
-  SMS / Call
-  Stakeholders / Customers

## DETAILED INCIDENT RESPONSE STEPS



### INCIDENT RESPONSE CHECKLIST



- ✓ Alert received and validated
- ✓ Incident created in tracking tool
- ✓ Severity and impact assessed
- ✓ Team notified and engaged
- ✓ Mitigation or rollback executed
- ✓ Service restored and validated
- ✓ Stakeholders communicated
- ✓ RCA completed
- ✓ Post-incident review done
- ✓ Action items tracked to closure

### ESCALATION MATRIX (EXAMPLE)

| TIME (MIN) | SEVERITY P1         | SEVERITY P2         | SEVERITY P3/P4      |
|------------|---------------------|---------------------|---------------------|
| 0-5        | On-Call Engineer    | On-Call Engineer    | Support Team        |
| 5-15       | Team Lead           | Team Lead           | Team Lead           |
| 15-30      | Engineering Manager | Engineering Manager | Team Lead           |
| 30+        | Director / CTO      | Director            | Engineering Manager |

### RCA FRAMEWORK (5 WHYS)

1. What happened?
  2. Why did it happen?
  3. Why did that happen?
  4. Why did that happen?
  5. Why did that happen?
- ↓  
Root Cause Identified



**TIP:**

The goal is not just to restore service, but to learn and make incidents less likely in the future.



# ENTERPRISE CASE STUDY

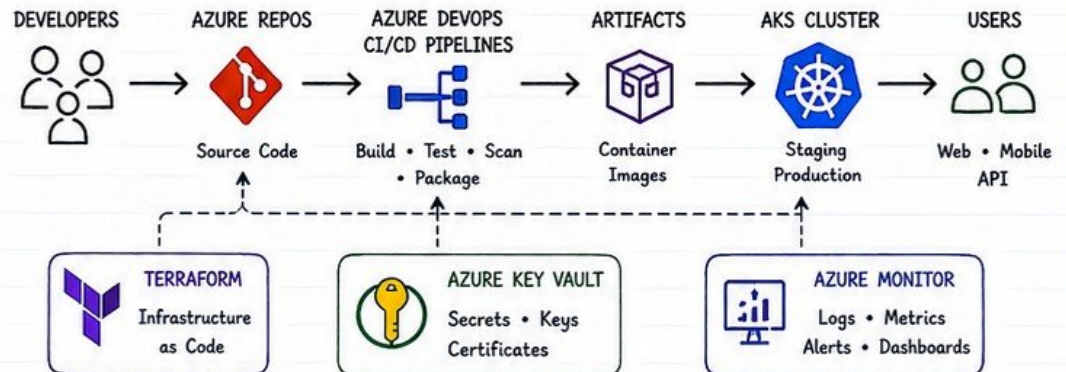
## BANKING PLATFORM CI/CD

AKS DEPLOYMENT • TERRAFORM • AZURE KEY VAULT • AZURE MONITOR • GITOPS • SECURITY

### OVERVIEW

A global bank modernized its core digital platform using Azure DevOps, Kubernetes (AKS), GitOps, and Infrastructure as Code to achieve faster delivery, strong security, and high reliability.

### SOLUTION ARCHITECTURE



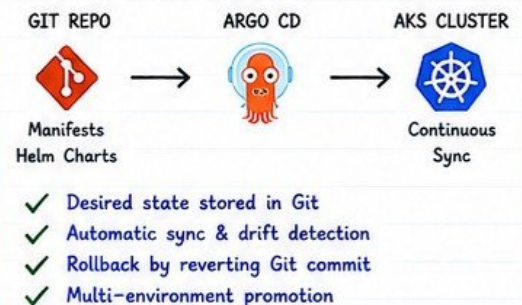
### TECHNOLOGY STACK

- Azure DevOps (Pipelines, Boards, Repos)
- Kubernetes (AKS)
- Terraform (IaC)
- Helm Charts
- Argo CD (GitOps)
- Azure Key Vault
- Azure Monitor & Log Analytics
- Azure Container Registry
- Microsoft Defender for Cloud

### CI/CD PIPELINE FLOW

1. Code Commit → Pull Request
2. Build & Unit Tests
3. Security Scan (SAST/DAST, Image Scan)
4. Package Docker Image
5. Push to Azure Container Registry
6. Deploy to AKS (Staging)
7. Integration & E2E Tests
8. Approval Gate
9. Deploy to AKS (Production)
10. Health Checks & Monitoring

### GITOPS WORKFLOW (ARGO CD)



### SECURITY IMPLEMENTATION

- Role-Based Access Control (RBAC)
- Managed Identities for workloads
- Secrets in Azure Key Vault
- Image scanning & vulnerability checks
- Network policies & NSGs
- Azure WAF & DDoS Protection
- Audit logs & compliance monitoring

### MONITORING & OBSERVABILITY

- Azure Monitor for metrics & alerts
- Log Analytics for centralized logs
- Application Insights for traces
- Dashboards for SRE & Dev teams
- Alerts via Email, Teams & SMS

### BUSINESS OUTCOMES

- ✓ 70% faster release cycle
- ✓ 99.95% platform availability
- ✓ Stronger security & compliance
- ✓ Reduced manual deployments
- ✓ Full auditability & traceability
- ✓ Multi-region ready architecture

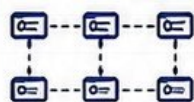
### LESSONS LEARNED

1. **AUTOMATION FIRST**  
Automate everything to reduce human error and increase speed.
2. **SECURITY BY DESIGN**  
Embed security in every stage of the pipeline.
3. **GIT IS THE SOURCE OF TRUTH**  
GitOps provides consistency and reliability.
4. **OBSERVABILITY MATTERS**  
You can't improve what you can't measure.
5. **PEOPLE & PROCESS**  
Tools don't solve problems—people and process do.

RECOGNIZE. AVOID. AUTOMATE. GOVERN. DELIVER VALUE.

## COMMON ENTERPRISE DEVOPS ANTI-PATTERNS

## THE IMPACT



### 1 PIPELINE SPRAWL

Too many pipelines doing similar things with inconsistent logic, duplicated steps, and no standards.

- High maintenance & duplication
- Inconsistent quality & security
- Hard to scale and govern



### 2 MANUAL DEPLOYMENTS

Relying on humans to deploy code, configure environments, or run release steps.

- Human errors & delays
- Inconsistent releases
- No audit trail



### 3 SHARED CREDENTIALS

Using shared accounts, hardcoded secrets, or storing credentials in code or configs.

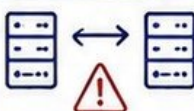
- Security risks & breaches
- No accountability
- Hard to rotate or revoke



### 4 LONG-RUNNING BUILDS

Slow, heavy builds that take too long, reduce feedback speed, and waste compute resources.

- Slow feedback loop
- Wasted compute cost
- Developer frustration



### 5 CONFIGURATION DRIFT

Environments drift over time due to manual changes, undocumented updates, or inconsistent configurations.

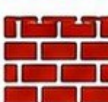
- Unexpected failures
- Compliance & audit issues
- Hard to troubleshoot



### 6 MONOLITHIC PIPELINES

One giant pipeline handling build, test, security, deploy, infra, and more without separation of concerns.

- Hard to troubleshoot
- Slow pipeline execution
- Difficult to reuse components



### 7 WEAK GOVERNANCE

No policies, standards, or guardrails. Everyone does things differently without oversight.

- Security & compliance risk
- Inconsistent practices
- No visibility or control



### 8 NO OBSERVABILITY IN DELIVERY

Lack of visibility into pipeline health, deployment status, change failure rate, or lead time.

- Issues go undetected
- Slow MTTR
- Poor user experience



## HOW TO AVOID THEM

### 1. STANDARDIZE PIPELINES



- Use templates
- Reuse components
- Apply conventions

### 2. AUTOMATE EVERYTHING



- IaC for infra
- GitOps for deploys
- Remove manual steps

### 3. SECURE BY DESIGN



- Use managed identities
- Store secrets in Key Vault
- Rotate credentials

### 4. OPTIMIZE BUILDS



- Parallel jobs
- Cache dependencies
- Build only what changed

### 5. ENFORCE CONSISTENCY



- Use IaC
- Drift detection
- Policy as Code

### 6. MODULARIZE PIPELINES



- Split by concern
- Use reusable stages
- Keep it simple

### 7. STRENGTHEN GOVERNANCE



- Define policies
- Use approvals
- Monitor & audit

### 8. INSTRUMENT & OBSERVE



- Pipeline metrics
- Alerts & dashboards
- Track DORA metrics



**TIP:**

Great DevOps is not about tools — it's about consistent practices, automation, security, and continuous improvement.



# SENIOR ENGINEER DESIGN PRINCIPLES

ENTERPRISE  
GOVERNANCE

PRINCIPLES THAT DRIVE ENTERPRISE-GRADE SYSTEMS.  
BUILT FOR SCALE. DESIGNED FOR IMPACT.

## 1 SCALABILITY



- Design for horizontal scaling.
- Stateless where possible.
- Decouple components.
- Handle exponential growth.
- Scale databases and queues independently.

## 2 RELIABILITY



- Build for high availability.
- Eliminate single points of failure.
- Implement retries, timeouts, and failovers.
- Test failure scenarios.
- Data integrity is non-negotiable.

## 3 MAINTAINABILITY



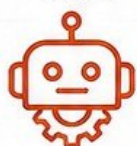
- Write clean, readable code.
- Modular and loosely coupled design.
- Document everything.
- Easy to debug and extend.
- Reduce complexity continuously.

## 4 SECURITY BY DESIGN



- Threat model early.
- Least privilege access.
- Encrypt data in transit and at rest.
- Validate all inputs.
- Automate security scans and compliance.

## 5 AUTOMATION FIRST



- Automate everything repeatable.
- CI/CD for every change.
- Infrastructure as Code.
- Self-service for teams.
- Reduce manual intervention to zero.

## 6 COST AWARENESS



- Optimize for value, not just speed.
- Right-size resources.
- Monitor and eliminate waste.
- Use managed services wisely.
- Build with FinOps mindset.

## 7 OBSERVABILITY



- You can't fix what you can't see.
- Centralized logs, metrics, traces.
- Alerts that matter.
- Correlate across the stack.
- Measure SLOs and SLIs.

## 8 PLATFORM THINKING



- Build internal platforms, not just apps.
- Standardize, abstract, and reuse.
- Empower teams with self-service.
- Consistency over customization.
- Developer experience is a product.

## THE SENIOR ENGINEER MINDSET



Think long-term



Ask why



Solve root causes



Elevate the team



Communicate clearly



Deliver business impact



Continuously improve



**TIP:**

Good systems follow frameworks.  
Great systems follow principles.





# AZURE DEVOPS PRODUCTION CHEAT SHEET

ESSENTIAL COMMANDS. REAL-WORLD USAGE. PRODUCTION READY.

## 1 ESSENTIAL AZURE CLI COMMANDS

|                                         |                           |
|-----------------------------------------|---------------------------|
| az login                                | Login to Azure            |
| az account set -s <sub-id>              | Set subscription          |
| az account list                         | List subscriptions        |
| az group list                           | List resource groups      |
| az group create -n rg-prod -l eastus    | Create resource group     |
| az resource list -g rg-prod             | List resources in RG      |
| az vm list -g rg-prod -o table          | List VMs                  |
| az storage account list -g rg-prod      | List storage accounts     |
| az keyvault list -g rg-prod             | List key vaults           |
| az acr list -g rg-prod                  | List container registries |
| az monitor metrics list --resource <id> | Get metrics               |
| az role assignment list --assignee <id> | List role assignments     |

## 2 PIPELINE COMMANDS (AZ DEVOPS CLI)

|                                                                        |                       |
|------------------------------------------------------------------------|-----------------------|
| az devops login                                                        | Login to Azure DevOps |
| az devops project list                                                 | List projects         |
| az pipelines list                                                      | List pipelines        |
| az pipelines show --id <id>                                            | Show pipeline         |
| az pipelines runs list --pipeline-id <id>                              | List runs             |
| az pipelines runs show --id <run-id>                                   | Show run details      |
| az pipelines runs queue --pipeline-id <id>                             | Queue pipeline        |
| az pipelines variable list --pipeline-id <id>                          | List variables        |
| az pipelines variable create --pipeline-id <id> --name <n> --value <v> | Create variable       |
| az repos list                                                          | List Git repos        |
| az repos show --id <repo-id>                                           | Show repo             |

## 3 GIT COMMANDS

|                                 |                   |
|---------------------------------|-------------------|
| git clone <repo-url>            | Clone repository  |
| git status                      | Check status      |
| git add .                       | Stage changes     |
| git commit -m "message"         | Commit changes    |
| git push origin <branch>        | Push changes      |
| git pull origin <branch>        | Pull latest       |
| git branch -a                   | List all branches |
| git checkout <branch>           | Switch branch     |
| git checkout -b <new-branch>    | Create & switch   |
| git log --oneline --graph --all | View history      |
| git tag -a v1.0.0 -m "release"  | Create tag        |
| git push origin --tags          | Push tags         |
| git stash                       | Stash changes     |
| git stash pop                   | Apply stash       |

## 4 AKS COMMANDS

|                                              |                   |
|----------------------------------------------|-------------------|
| az aks list -g <rg> -o table                 | List AKS clusters |
| az aks get-credentials -g <rg> -n <aks>      | Get kubeconfig    |
| kubectl get nodes                            | List nodes        |
| kubectl get pods -A                          | List all pods     |
| kubectl get deployments -A                   | List deployments  |
| kubectl get svc -A                           | List services     |
| kubectl describe pod <pod>                   | Describe pod      |
| kubectl logs <pod> -n <ns>                   | Pod logs          |
| kubectl top nodes                            | Node metrics      |
| kubectl top pods -A                          | Pod metrics       |
| kubectl get events -A                        | Recent events     |
| --sort-by=.lastTimestamp                     |                   |
| kubectl delete pod <pod> -n <ns>             | Delete pod        |
| kubectl rollout status deploy/<name> -n <ns> | Rollout status    |

## 5 HELM COMMANDS

|                                                             |                   |
|-------------------------------------------------------------|-------------------|
| helm repo add bitnami https://charts.bitnami.com/bitnami    | Add repo          |
| helm repo list                                              | List repos        |
| helm repo update                                            | Update repos      |
| helm search repo nginx                                      | Search charts     |
| helm install myapp bitnami/nginx -n prod --create-namespace | Install chart     |
| helm list -A                                                | List releases     |
| helm status myapp -n prod                                   | Release status    |
| helm upgrade myapp bitnami/nginx -n prod                    | Upgrade release   |
| helm rollback myapp 1 -n prod                               | Rollback release  |
| helm uninstall myapp -n prod                                | Uninstall release |
| helm history myapp -n prod                                  | Release history   |
| helm lint ./chart                                           | Lint chart        |

## 6 TERRAFORM COMMANDS

|                                   |                     |
|-----------------------------------|---------------------|
| terraform --version               | Check version       |
| terraform init                    | Initialize          |
| terraform validate                | Validate config     |
| terraform fmt                     | Format files        |
| terraform plan                    | Show execution plan |
| terraform plan -out=tfplan        | Save execution plan |
| terraform apply                   | Apply changes       |
| terraform apply tfplan            | Apply saved plan    |
| terraform show                    | Show state          |
| terraform destroy                 | Destroy resources   |
| terraform workspace list          | List workspaces     |
| terraform workspace new <name>    | Create workspace    |
| terraform workspace select <name> | Switch workspace    |
| terraform output                  | Show outputs        |

## 7 TROUBLESHOOTING COMMANDS

|                                                                                                                        |                   |
|------------------------------------------------------------------------------------------------------------------------|-------------------|
| az vm get-instance-view -g <rg> -n <vm> -o table                                                                       | VM status         |
| az resource show -g <rg> -n <name> -o json                                                                             | Resource details  |
| az monitor activity-log list --max-items 200                                                                           | Activity log      |
| az monitor metrics list --resource <id> --metric <name>                                                                | Metrics           |
| kubectl describe pod <pod> -n <ns>                                                                                     | Pod details       |
| kubectl logs <pod> -n <ns> --previous                                                                                  | Previous logs     |
| kubectl get events -A --field-selector type=Warning                                                                    | Warning events    |
| az acr repository list -n <acr> -o table                                                                               | List images       |
| az acr repository show-tags -n <acr> -r <repo> -o table                                                                | Image tags        |
| az network watcher test-connectivity -g <rg> --source-resource <id> --dest-resource <id> --protocol Tcp --dest-port 80 | Test connectivity |
| nslookup <domain>                                                                                                      | DNS lookup        |
| curl -I <url>                                                                                                          | Check endpoint    |

## 8 DEPLOYMENT CHECKLIST



- |                                                             |                                                             |
|-------------------------------------------------------------|-------------------------------------------------------------|
| <input type="checkbox"/> Code reviewed and approved         | <input type="checkbox"/> RBAC and access reviewed           |
| <input type="checkbox"/> Build and unit tests passing       | <input type="checkbox"/> Resource limits and quotas checked |
| <input type="checkbox"/> Security scans passed              | <input type="checkbox"/> Rollback plan validated            |
| <input type="checkbox"/> Artifacts stored in secure feed    | <input type="checkbox"/> Staging tests passed               |
| <input type="checkbox"/> Infrastructure plan reviewed       | <input type="checkbox"/> Performance tests passed           |
| <input type="checkbox"/> Secrets stored in Key Vault        | <input type="checkbox"/> Change ticket created              |
| <input type="checkbox"/> Configuration validation completed | <input type="checkbox"/> Stakeholders notified              |
| <input type="checkbox"/> Backup strategy verified           | <input type="checkbox"/> Production deployment approved     |
| <input type="checkbox"/> Monitoring & alerts configured     | <input type="checkbox"/> Go-Live communication prepared     |



**TIP:**

Automate everything.  
Validate continuously.  
Monitor relentlessly.  
Document always.  
Improve continuously.

**THAT'S HOW WE  
STAY IN PRODUCTION.**

**VERIQTA**





**VERIQTA**  
• ENGINEERED FOR IMPACT •



# ENTERPRISE PRODUCTION CHECKLIST

ENTERPRISE  
GOVERNANCE



EVERY CHECK. EVERY TIME. PRODUCTION EXCELLENCE.  
ZERO SURPRISES.

1

## PIPELINE READINESS



- ☐ Code merged and reviewed
- ☐ Build successful
- ☐ All automated tests passing
- ☐ Security scans passed
- ☐ Artifacts signed and stored
- ☐ Version tags created
- ☐ Pipeline run completed
- ☐ Release notes generated
- ☐ Approval gates passed

2

## SECURITY VALIDATION



- ☐ Secrets stored in Key Vault
- ☐ No hardcoded credentials
- ☐ Least privilege access
- ☐ Dependencies scanned
- ☐ Container image scanned
- ☐ Network policies applied
- ☐ RBAC roles validated
- ☐ Encryption in transit enabled
- ☐ Security sign-off completed

3

## INFRASTRUCTURE VALIDATION



- ☐ IaC plan reviewed and approved
- ☐ Terraform/ARM applied
- ☐ Resources created successfully
- ☐ Tags and naming compliant
- ☐ Backup configured
- ☐ High availability enabled
- ☐ Cost estimates reviewed
- ☐ Quota and limits checked
- ☐ Infra sign-off completed

4

## KUBERNETES VALIDATION



- ☐ Cluster healthy
- ☐ Nodes Ready
- ☐ System pods running
- ☐ Workloads deployed
- ☐ Liveness & readiness probes set
- ☐ Resource requests/limits set
- ☐ HPA configured (if needed)
- ☐ Network policies validated
- ☐ K8s sign-off completed

5

## COMPLIANCE REVIEW



- ☐ Policy as Code validated
- ☐ Compliance controls met
- ☐ Audit evidence updated
- ☐ Data classification verified
- ☐ Retention policies enforced
- ☐ Access reviews completed
- ☐ Regulatory requirements met
- ☐ Compliance sign-off completed
- ☐ Documentation updated

6

## MONITORING VERIFICATION



- ☐ Logs collected and centralized
- ☐ Metrics dashboards verified
- ☐ Alerts configured and tested
- ☐ SLOs and SLIs defined
- ☐ Synthetic tests passing
- ☐ Health checks verified
- ☐ On-call notifications tested
- ☐ Runbooks linked in alerts
- ☐ Monitoring sign-off completed

7

## ROLLBACK READINESS



- ☐ Rollback strategy documented
- ☐ Previous version available
- ☐ DB rollback plan tested
- ☐ Feature flags configured
- ☐ Blue/Green or Canary ready
- ☐ Rollback pipeline tested
- ☐ Data backup verified
- ☐ RTO and RPO validated
- ☐ Rollback sign-off completed

8

## DEPLOYMENT APPROVAL



- ☐ All approvals collected
- ☐ Business owner approved
- ☐ Security approved
- ☐ Operations approved
- ☐ Compliance approved
- ☐ Change ticket updated
- ☐ Maintenance window confirmed
- ☐ Stakeholders notified
- ☐ Go-Live approved

## FINAL GO-LIVE CONFIRMATION



- ☐ All checklist items completed
- ☐ No open critical or high issues
- ☐ Stakeholders informed
- ☐ Team on standby
- ☐ Go-Live authorized

## LEGEND

- ☐ Complete
- ☐ In Progress
- ☐ Not Started
- ☐ Not Applicable



## GOAL:

Deploy with confidence.  
Operate with excellence.  
Deliver continuous value.



**TIP:**

A good checklist prevents incidents.  
A great checklist prevents disasters.



**VERIQTA**



YouTube  
/veriqta



GitHub  
/veriqta



LinkedIn  
/company/veriqta



X  
@veriqta



Instagram  
@veriqta\_academy



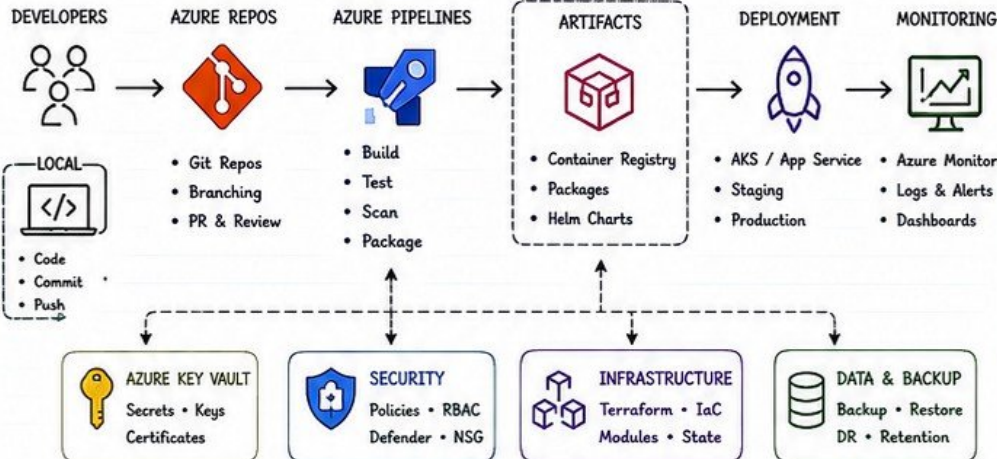
Telegram  
t.me/veriqta

# SENIOR PRODUCTION REVIEW

REVIEW. REFINE. RAISE THE STANDARD. DELIVER EXCELLENCE.

ENTERPRISE  
GOVERNANCE

## END-TO-END AZURE DEVOPS ARCHITECTURE



## ENTERPRISE WORKFLOW SUMMARY

- 1 PLAN**  
Requirements • Design • Backlog
- 2 CODE**  
Develop • Commit • Pull Request
- 3 BUILD & TEST**  
Build • Unit Test • Security Scan
- 4 PACKAGE**  
Artifact • Image • Helm Chart
- 5 DEPLOY**  
Staging • Production • Blue/Green
- 6 MONITOR**  
Logs • Metrics • Alerts • Dashboards
- 7 OPERATE & IMPROVE**  
Feedback • Optimize • Automate

## PRODUCTION READINESS SCORECARD

| CATEGORY                   | WEIGHT      | SCORE (0-5)    | WEIGHTED         |
|----------------------------|-------------|----------------|------------------|
| Reliability & Availability | 20%         | ___            | ___              |
| Security & Compliance      | 20%         | ___            | ___              |
| Performance & Scalability  | 15%         | ___            | ___              |
| Automation & CI/CD         | 15%         | ___            | ___              |
| Observability & Monitoring | 10%         | ___            | ___              |
| Backup & DR Readiness      | 10%         | ___            | ___              |
| Operational Excellence     | 10%         | ___            | ___              |
| <b>TOTAL</b>               | <b>100%</b> | <b>___ / 5</b> | <b>___ / 100</b> |

## COMMON PRODUCTION FAILURES

- ✗ Poor requirements & design
- ✗ No automated tests
- ✗ Manual deployments
- ✗ Insufficient monitoring & alerting
- ✗ Weak access control & secrets management
- ✗ Configuration drift
- ✗ Resource limits not set
- ✗ No rollback strategy
- ✗ Ignoring logs & warnings
- ✗ Lack of documentation
- ✗ No chaos testing / failure practice
- ✗ Scaling issues discovered in production

## SENIOR ENGINEER LESSONS

- ✓ Design for failure, not for success.
- ✓ Automate everything that can be automated.
- ✓ Security is a feature, not an afterthought.
- ✓ Observability is your superpower.
- ✓ Keep it simple, but not too simple.
- ✓ Measure everything. Guess nothing.
- ✓ Document for today, train for tomorrow.
- ✓ Pay down technical debt continuously.
- ✓ Ownership never stops at deployment.
- ✓ Think long-term. Build with purpose.

## ARCHITECTURE REVIEW CHECKLIST

- ☐ Business requirements & SLAs understood
- ☐ High-level architecture documented
- ☐ Security architecture reviewed
- ☐ IaC modules reviewed & reusable
- ☐ Networking & connectivity validated
- ☐ Identity & access model reviewed
- ☐ Data flow & storage reviewed
- ☐ Resiliency & DR strategy validated
- ☐ Cost estimation completed
- ☐ Risks & mitigations identified
- ☐ Stakeholder sign-off obtained
- ☐ Documentation updated



## OPERATIONAL EXCELLENCE CHECKLIST

- ☐ Runbooks & SOPs available
- ☐ On-call rotation defined & tested
- ☐ Alerts actionable & noise-free
- ☐ Metrics, logs & traces available
- ☐ Incident response tested
- ☐ Capacity & performance reviewed
- ☐ Patch & update process defined
- ☐ Backup & restore tested
- ☐ Cost & usage reviewed
- ☐ Service health dashboards active
- ☐ Post-incident reviews completed
- ☐ Continuous improvement in progress



## NEXT LEARNING PATH

### ENTERPRISE PLATFORM ENGINEERING AND MULTI-CLOUD DEVOPS



#### Focus Areas:

- Internal Developer Platforms (IDP)
- Multi-Cloud CI/CD & GitOps
- Service Mesh & Advanced Networking
- FinOps & Cost Optimization
- Policy as Code & Governance at Scale
- Chaos Engineering & Resiliency at Scale



**TIP:**

A senior engineer doesn't just solve problems. They prevent them, learn from them, and raise the standard for everyone.

